

SQLD 실전형 예상문제 50 제

1 과목 데이터 모델링의 이해

문제 01~10 은 데이터 모델링, 엔터티/속성/관계/식별자, 정규화와 반정규화 중심입니다.

문제 01. [데이터모델의 이해 - 스키마] 다음 설명에 해당하는 스키마로 가장 적절한 것은?

[설명]

조직 전체 관점에서 모든 사용자 관점을 통합하여 표현하며, 데이터베이스 전체의 논리적 구조와 제약조건을 정의한다.

- ① 외부 스키마
- ② 개념 스키마
- ③ 내부 스키마
- ④ 서브 스키마

정답: ② 개념 스키마

입문자용 상세 해설

3 단계 스키마에서 “조직 전체 관점”과 “데이터베이스 전체의 논리적 구조”가 나오면 개념 스키마를 떠올리면 된다. 개념 스키마는 개별 화면이나 사용자별 요구가 아니라, 모든 사용자 관점을 통합한 전체 데이터 구조와 제약조건을 표현한다.

풀이 흐름

- 외부 스키마는 사용자별 관점이다. 예를 들어 학생은 성적 조회 화면, 교직원은 성적 입력 화면처럼 같은 DB 라도 바라보는 화면이 다를 수 있다.
- 개념 스키마는 조직 전체의 통합 논리 모델이다. “우리 조직의 학생, 과목, 수강, 성적이 어떤 구조로 연결되는가”를 설명한다.
- 내부 스키마는 저장 장치 관점이다. 인덱스, 저장 위치, 파일 구조처럼 물리적 저장 방법에 가깝다.
-

흐름 도식

사용자별 화면/업무 관점

↓ 통합

개념 스키마: 전체 논리 구조와 제약조건

↓ 구현

내부 스키마: 물리적 저장 구조

비교 정리

구분	관점	핵심 표현	예시
외부 스키마	개별 사용자/응용 프로그램	사용자 관점, 서브 스키마	학생용 성적 조회 화면
개념 스키마	조직 전체	전체 논리 구조, 통합 관점	학생-수강-과목 전체 구조
내부 스키마	DBMS/저장 장치	물리적 저장 구조	인덱스, 파일 배치

함정 포인트

문제의 “논리적 구조”라는 단어만 보고 임의로 “논리 스키마”라는 용어를 만들면 안 된다. SQLD 의 기본 구분은 외부/개념/내부 스키마다.

문제 02. [엔터티 - 발생 시점에 따른 분류] 다음 중 엔터티를 발생 시점에 따라 분류한 유형으로 적절하지 않은 것은?

- ① 기본 엔터티
- ② 중심 엔터티
- ③ 행위 엔터티
- ④ 관계 엔터티

정답: ④ 관계 엔터티

입문자용 상세 해설

엔터티를 발생 시점에 따라 분류하면 기본 엔터티, 중심 엔터티, 행위 엔터티로 나눈다. 관계 엔터티는 관계를 하나의 엔터티로 승격시킨 경우를 설명하는 표현이지, 발생 시점 분류의 공식 세트는 아니다.

풀이 흐름

- 기본 엔터티는 다른 엔터티의 영향을 크게 받지 않고 독립적으로 존재하는 기초 데이터다. 고객, 상품, 부서처럼 기준 정보가 대표적이다.
- 중심 엔터티는 기본 엔터티로부터 발생하고 업무의 중심 흐름을 만든다. 주문, 계약, 수강신청처럼 업무 처리의 중심이 된다.
- 행위 엔터티는 두 개 이상의 엔터티 관계에서 실제 행위나 이력을 표현한다. 주문상세, 출고이력, 결제이력 등이 여기에 가깝다.
-

흐름 도식

기본 엔터티(기준 정보) → 중심 엔터티(주요 업무) → 행위 엔터티(상세 행위/이력)

비교 정리

분류	쉽게 말하면	예시
기본 엔터티	업무의 기준이 되는 마스터 데이터	고객, 상품, 부서
중심 엔터티	업무 흐름의 중심이 되는 데이터	주문, 계약, 수강신청
행위 엔터티	업무 처리 결과나 이력을 남기는 데이터	주문상세, 결제이력, 배송이력
관계 엔터티	관계를 별도 엔터티로 만든 것	학생-과목 사이의 수강

함정 포인트

“관계 엔터티”라는 말 자체는 실무에서 쓰일 수 있지만, 이 문제는 “발생 시점에 따른 분류”를 묻고 있으므로 기본/중심/행위만 기억해야 한다.

문제 03. [속성 - 속성의 분류] 학생 엔터티에 생년월일 속성이 저장되어 있고, 화면에는 생년월일을 이용하여 계산한 만 나이를 보여준다. 이때 “나이” 속성의 분류로 가장 적절한 것은?

- ① 기본 속성
- ② 설계 속성
- ③ 파생 속성
- ④ 다중값 속성

정답: ③ 파생 속성

입문자용 상세 해설

나이는 생년월일이라는 다른 속성으로부터 계산된다. 따라서 “저장된 원천값”이 아니라 “계산으로 얻은 값”이므로 파생 속성이다.

풀이 흐름

- 생년월일은 원천 데이터다. 사람이 태어난 날짜는 업무적으로 직접 저장할 수 있는 값이다.
- 나이는 현재 날짜와 생년월일을 이용해 계산한다. 시간이 지나면 값이 계속 바뀐다.
- 계산 결과를 화면에 보여주거나 성능 때문에 저장할 수는 있지만, 분류상으로는 파생 속성이다.

흐름 도식

생년월일 + 현재 날짜
↓ 계산
나이 = 파생 속성

비교 정리

속성 분류	의미	예시
기본 속성	업무상 원래 존재하는 속성	생년월일, 이름, 전화번호
설계 속성	모델 설계 과정에서 추가한 속성	인공 식별자, 순번
파생 속성	다른 속성으로 계산한 속성	나이, 총주문금액, 평균점수
다중값 속성	하나의 인스턴스가 여러 값을 가질 수 있는 속성	여러 연락처, 여러 이메일

함정 포인트

업무에서 자주 쓴다고 모두 기본 속성이 되는 것은 아니다. “어떻게 만들어지는 값인가”를 기준으로 판단해야 한다.

문제 04. [속성 - 단순/복합/단일값/다중값] 속성의 특성에 대한 설명으로 옳지 않은 것은?

- ① 주소가 시, 군, 구, 상세주소 등으로 분해될 수 있다면 복합 속성으로 볼 수 있다.
- ② 한 고객이 여러 개의 연락처를 가질 수 있다면 연락처는 다중값 속성으로 볼 수 있다.
- ③ 주민등록번호처럼 한 인스턴스가 하나의 값만 갖는 속성은 단일값 속성으로 볼 수 있다.
- ④ 생년월일로 계산되는 나이는 업무에서 사용되므로 기본 속성으로만 분류된다.

정답: ④ 생년월일로 계산되는 나이는 업무에서 사용되므로 기본 속성으로만 분류된다.

입문자용 상세 해설

속성 분류는 한 가지 기준만 있는 것이 아니다. 주소처럼 여러 하위 요소로 나눌 수 있는지는 단순/복합 기준이고, 연락처처럼 여러 값을 가질 수 있는지는 단일값/다중값 기준이다. 나이처럼 다른 값으로 계산되는지는 기본/파생 기준이다.

풀이 흐름

- 주소는 시, 군, 구, 상세주소 등으로 나눌 수 있으므로 복합 속성으로 볼 수 있다.
- 한 고객이 연락처를 여러 개 가질 수 있다면 연락처는 다중값 속성으로 볼 수 있다.
- 주민등록번호처럼 한 사람에게 하나의 값만 있다면 단일값 속성으로 볼 수 있다.
- 생년월일로 계산되는 나이는 업무에서 쓰이더라도 파생 속성이다.

비교 정리

분류 기준	종류	판단 질문
구성 여부	단순/복합 속성	하나의 값을 더 작은 의미 단위로 나눌 수 있는가?
값의 개수	단일값/다중값 속성	하나의 인스턴스가 값을 하나만 갖는가, 여러 개 갖는가?
생성 방식	기본/설계/파생 속성	원래 존재하는가, 설계상 추가했는가, 계산했는가?

함정 포인트

속성 분류는 서로 다른 축으로 동시에 판단할 수 있다. 예를 들어 “연락처”는 다중값 속성이면서, 세부적으로 휴대폰/집전화로 나누면 복합적으로 다룰 수도 있다.

문제 05. [식별자 - 주식별자 특징] 주식별자에 대한 설명으로 옳지 않은 것은?

- ① 하나의 인스턴스를 유일하게 식별할 수 있어야 한다.
- ② 주식별자는 최소성, 유일성, 불변성 등을 고려하여 선정한다.
- ③ 주식별자 속성은 NULL 값을 허용하지 않는 것이 원칙이다.
- ④ 아직 값이 확정되지 않은 경우에는 주식별자에 NULL 값을 저장할 수 있다.

정답: ④ 아직 값이 확정되지 않은 경우에는 주식별자에 NULL 값을 저장할 수 있다.

입문자용 상세 해설

주식별자는 한 행을 정확히 찾기 위한 대표 식별자다. 따라서 값이 없어도 되는 NULL 은 허용하면 안 된다. 아직 값이 확정되지 않는 속성은 주식별자로 선택하기 어렵다.

풀이 흐름

- 유일성: 같은 값으로 두 인스턴스가 식별되면 안 된다.
- 최소성: 식별에 꼭 필요한 속성만 포함해야 한다.
- 불변성: 값이 자주 바뀌면 참조하는 관계가 흔들린다.
- 존재성/NOT NULL: 식별자가 비어 있으면 해당 인스턴스를 찾을 수 없다.
-

흐름 도식

주식별자 선택 기준

유일한가? → 최소한인가? → 변하지 않는가? → NULL 이 아닌가?

비교 정리

특징	의미	위반 예시
유일성	각 인스턴스를 하나로 구분	동명이인을 이름만으로 식별
최소성	불필요한 속성을 제외	학번만으로 충분한데 학번+이름 사용
불변성	값이 안정적이어야 함	변경 가능한 전화번호를 주식별자로 사용
존재성	NULL 이면 안 됨	아직 발급되지 않은 번호를 식별자로 사용

함정 포인트

“나중에 값이 들어온다”는 이유는 주식별자 NULL 허용 근거가 아니다. 이런 경우에는 인공 식별자를 두거나 다른 후보 식별자를 검토한다.

문제 06. [관계 - 관계차수와 선택사양] 다음 업무 규칙을 ERD 로 표현한다고 할 때, 설명으로 옳지 않은 것은?

[업무 규칙]

- 한 고객은 주문을 하지 않을 수도 있고, 여러 건의 주문을 할 수도 있다.

- 한 주문은 반드시 한 명의 고객에 의해 발생한다.

- ① 고객과 주문은 1:N 관계로 볼 수 있다.
- ② 고객 입장에서 주문은 선택 관계일 수 있다.
- ③ 주문 엔터티에는 고객의 식별자가 외래식별자로 포함될 수 있다.
- ④ 고객과 주문은 N:M 관계이므로 반드시 별도의 교차 엔터티가 필요하다.

정답: ④ 고객과 주문은 N:M 관계이므로 반드시 별도의 교차 엔터티가 필요하다.

입문자용 상세 해설

고객 한 명은 주문을 0 건 이상 가질 수 있고, 주문 한 건은 반드시 고객 한 명에게 속한다. 따라서 고객:주문은 1:N 관계이며, 주문 쪽에 고객 식별자가 외래키로 들어가는 구조가 자연스럽다.

풀이 흐름

- 고객 기준: 주문을 하지 않을 수도 있으므로 최소 0, 여러 건 가능하므로 최대 N 이다.
- 주문 기준: 한 주문은 반드시 한 명의 고객에 의해 발생하므로 최소 1, 최대 1 이다.
- 따라서 전체 관계는 고객 1 명 대 주문 여러 건, 즉 1:N 이다.

흐름 도식

고객 1 명 — 0..N 주문
주문 1 건 — 반드시 1 명 고객

비교 정리

관점	최소	최대	의미
고객 → 주문	0	N	고객은 주문이 없을 수도 있고 여러 건 할 수도 있음
주문 → 고객	1	1	주문은 반드시 한 고객에게 속함

함정 포인트

“여러 건의 주문”이라는 표현만 보고 N:M 이라고 판단하면 안 된다. 반드시 반대 방향도 확인해야 한다. 주문 한 건이 여러 고객에게 속하지 않으므로 N:M 이 아니다.

문제 07. [식별자 관계와 비식별자 관계] 주문상세 엔터티의 주식별자가 (주문번호, 주문순번)이고, 주문번호는 주문 엔터티의 주식별자를 상속받은 외래식별자다. 이 관계에 대한 설명으로 가장 적절한 것은?

- ① 주문과 주문상세는 식별자 관계로 볼 수 있다.
- ② 주문과 주문상세는 비식별자 관계이므로 주문번호는 주문상세의 일반 속성이다.
- ③ 주문상세는 주문과 독립적으로 존재하므로 주문번호를 가질 수 없다.
- ④ 식별자 관계에서는 부모의 식별자가 자식 엔터티로 전달되지 않는다.

정답: ① 주문과 주문상세는 식별자 관계로 볼 수 있다.

입문자용 상세 해설

식별자 관계의 핵심은 부모의 주식별자가 자식의 주식별자 일부가 되는지 여부다. 주문상세의 주식별자가 (주문번호, 주문순번)이고 주문번호가 부모 주문의 주식별자라면, 부모 식별자가 자식 식별자에 포함되므로 식별자 관계다.

풀이 흐름

- 주문 엔터티의 주식별자: 주문번호
- 주문상세 엔터티의 주식별자: 주문번호 + 주문순번
- 부모의 주문번호가 자식의 주식별자 일부가 되었으므로 식별자 관계로 판단한다.

흐름 도식

주문(주문번호 PK)
| 주문번호 전달
▼
주문상세(PK = 주문번호 + 주문순번)

비교 정리

관계 유형	부모 식별자가 자식에게 전달되는 방식	예시
식별자 관계	자식의 주식별자 일부가 됨	주문상세 PK = 주문번호 + 순번
비식별자 관계	자식의 일반 외래키가 됨	사원 테이블의 부서 ID 는 FK 이지만 사원 PK 는 사원 ID

함정 포인트

외래키가 있다고 무조건 비식별자 관계가 아니다. “외래키가 자식의 PK 안에 포함되는가”가 핵심 기준이다.

문제 08. [정규화 - 제 1 정규형] 다음 고객 테이블은 연락처가 반복 컬럼으로 설계되어 있다. 제 1 정규형 관점에서 가장 적절한 개선 방안은?

[고객]

고객 ID | 고객명 | 연락처 1 | 연락처 2 | 연락처 3

- ① 연락처 4, 연락처 5 컬럼을 추가하여 확장성을 높인다.
- ② 연락처 1 만 남기고 나머지 연락처는 삭제한다.
- ③ 고객연락처 엔터티를 분리하여 고객 ID, 연락처순번, 연락처번호 형태로 관리한다.
- ④ 고객명 컬럼을 연락처 테이블로 이동하고 고객 테이블을 삭제한다.

정답: ③ 고객연락처 엔터티를 분리하여 고객 ID, 연락처순번, 연락처번호 형태로 관리한다.

입문자용 상세 해설

제 1 정규형의 핵심은 컬럼 하나에 원자값을 두고, 반복 컬럼을 제거하는 것이다. 연락처 1, 연락처 2, 연락처 3 처럼 같은 의미의 컬럼이 반복되면 연락처를 별도 엔터티로 분리하는 것이 적절하다.

풀이 흐름

- 현재 구조는 연락처가 몇 개까지 가능한지 컬럼 수로 제한된다.
- 연락처 4, 연락처 5 를 추가하면 일시적으로 해결될 수 있지만 같은 문제가 반복된다.
- 고객연락처 엔터티를 만들면 한 고객이 연락처를 0 개, 1 개, 여러 개 가질 수 있다.

흐름 도식

정규화 전: 고객(고객 ID, 고객명, 연락처 1, 연락처 2, 연락처 3)

↓ 반복 컬럼 분리

정규화 후: 고객(고객 ID, 고객명) + 고객연락처(고객 ID, 연락처순번, 연락처번호)

비교 정리

방식	장점/문제점
반복 컬럼 추가	컬럼 수가 계속 늘고, 검색/집계/제약조건 관리가 어려움
별도 엔터티 분리	연락처 개수 변화에 유연하고, 하나의 연락처를 한 행으로 관리 가능

함정 포인트

정규화는 단순히 컬럼 수를 줄이는 작업이 아니다. 데이터가 반복되는 구조를 더 안정적인 행 구조로 바꾸는 작업이다.

문제 09. [정규화 - 제 3 정규형] 다음 사원 테이블에서 함수 종속이 아래와 같을 때, 제거해야 할 종속 관계와 관련된 정규형으로 가장 적절한 것은?

[사원]

사원 ID, 사원명, 부서 ID, 부서명

[함수 종속]

사원 ID → 사원명, 부서 ID

부서 ID → 부서명

- ① 제 1 정규형 - 반복 속성 제거
- ② 제 2 정규형 - 부분 함수 종속 제거
- ③ 제 3 정규형 - 이행 함수 종속 제거
- ④ 제 4 정규형 - 다치 종속 제거

정답: ③ 제 3 정규형 - 이행 함수 종속 제거

입문자용 상세 해설

사원 ID 가 부서 ID 를 결정하고, 부서 ID 가 부서명을 결정한다. 결국 사원 ID 가 부서명을 간접적으로 결정하므로 이행 함수 종속이다. 이행 함수 종속 제거는 제 3 정규형의 핵심 내용이다.

풀이 흐름

- 사원 ID → 사원명, 부서 ID: 사원을 알면 사원명과 소속 부서를 알 수 있다.
- 부서 ID → 부서명: 부서 ID 를 알면 부서명을 알 수 있다.
- 따라서 사원 ID → 부서 ID → 부서명 흐름이 생긴다. 부서명은 사원 ID 에 직접 종속된 것처럼 보이지만 실제로는 부서 ID 를 거쳐 결정된다.
- 분리하면 사원(사원 ID, 사원명, 부서 ID)과 부서(부서 ID, 부서명)가 된다.

흐름 도식

사원 ID → 부서 ID → 부서명
↑
이 구조가 이행 함수 종속

비교 정리

정규형	제거 대상	입문자용 설명
1NF	반복 속성, 비원자값	한 칸에 여러 값을 넣지 말자
2NF	부분 함수 종속	복합키의 일부에만 종속되는 일반 속성을 분리하자
3NF	이행 함수 종속	일반 속성이 다른 일반 속성을 결정하는 구조를 분리하자
BCNF	모든 결정자가 후보키가 아닌 종속	결정자 역할을 하는 속성은 후보키여야 한다
4NF	다치 종속	서로 독립적인 다중값 정보를 한 테이블에 섞지 말자

함정 포인트

원본 보기 ④의 “BCNF - 다치 종속 제거”는 개념 매핑이 부정확하다. 다치 종속은 일반적으로 제 4 정규형과 연결된다. 이 문항의 정답 ③은 그대로 맞다.

문제 10. [반정규화와 성능 데이터 모델링] 반정규화 적용 대상으로 가장 부적절한 것은?

- ① 대량 범위 처리가 빈번하고 조인 비용이 매우 큰 경우
- ② 업무 규칙이 확정되지 않았으므로 임의로 중복 컬럼을 먼저 추가하는 경우
- ③ 통계성 데이터를 반복적으로 계산하여 성능 저하가 큰 경우
- ④ 다수 테이블 조인이 빈번하여 특정 조회 성능이 심각하게 저하되는 경우

정답: ② 업무 규칙이 확정되지 않았으므로 임의로 중복 컬럼을 먼저 추가하는 경우

입문자용 상세 해설

반정규화는 성능을 위해 의도적으로 중복을 허용하는 설계 기법이다. 단, 업무 규칙과 정합성 관리 방법이 명확할 때 적용해야 한다. 업무 규칙이 확정되지 않은 상태에서 중복 컬럼을 먼저 추가하는 것은 반정규화가 아니라 위험한 임의 설계에 가깝다.

풀이 흐름

- 정규화는 데이터 중복을 줄이고 정합성을 높인다.
- 하지만 조회가 매우 빈번하고 조인 비용이 크면 정규화된 구조가 성능 병목이 될 수 있다.
- 이때 중복 컬럼, 집계 테이블, 이력 요약 테이블 등을 통제된 방식으로 추가할 수 있다.
- 반정규화 후에는 중복 데이터가 서로 다르지 않도록 갱신 규칙과 검증 방법이 필요하다.

흐름 도식

정규화 모델 완성 → 성능 병목 확인 → 정합성 영향 분석 → 반정규화 적용 → 동기화/검증 관리

비교 정리

적용하기 좋은 경우	주의해야 할 경우
대량 조인 때문에 응답 시간이 심각하게 느림	업무 규칙이 아직 불명확함
반복 계산되는 통계/합계가 많음	단순히 개발이 귀찮아서 중복 컬럼을 추가함
조회 성능이 핵심이고 갱신 규칙이 명확함	중복 데이터 불일치를 관리할 방법이 없음

함정 포인트

반정규화는 “정규화를 안 하는 것”이 아니다. 정규화된 모델을 기준으로 성능과 정합성 비용을 비교한 뒤 선택하는 설계 판단이다.

2 과목 SQL 기본 및 활용

문제 11. [SELECT 문 - 논리적 실행 순서] 다음 중 SELECT 문의 논리적 실행 순서로 가장 적절한 것은?

- ① SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY
- ② WHERE → FROM → GROUP BY → HAVING → SELECT → ORDER BY
- ③ FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
- ④ FROM → WHERE → HAVING → GROUP BY → SELECT → ORDER BY

정답: ③ FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

입문자용 상세 해설

SQL 은 작성할 때 SELECT 를 맨 앞에 쓰지만, 논리적으로는 FROM 에서 조회 대상을 만든 뒤 WHERE, GROUP BY, HAVING, SELECT, ORDER BY 순서로 처리한다.

풀이 흐름

- FROM: 어느 테이블에서 데이터를 가져올지 결정하고 조인 결과 집합을 만든다.
- WHERE: 개별 행 단위 조건으로 필요 없는 행을 제거한다.
- GROUP BY: 남은 행을 그룹으로 묶는다.
- HAVING: 그룹 결과에 조건을 적용한다.
- SELECT: 최종적으로 보여줄 컬럼과 계산식을 만든다.
- ORDER BY: 결과를 정렬한다.
-

흐름 도식

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

비교 정리

절	처리 대상	예시
WHERE	그룹화 전의 개별 행	SAL > 3000
GROUP BY	행을 묶는 기준	DEPTNO 별 그룹
HAVING	그룹화 후의 집계 결과	COUNT(*) >= 3
ORDER BY	최종 결과	SUM(SAL) DESC

함정 포인트

SELECT 별칭은 논리적으로 SELECT 단계에서 만들어진다. 따라서 같은 SELECT 블록의 WHERE 에서 별칭을 바로 쓰기 어렵고, ORDER BY 에서는 사용할 수 있는 경우가 많다.

문제 12. [WHERE 절 - 논리 연산자 우선순위] 다음 테이블과 SQL 의 실행 결과 COUNT(*) 값으로 옳은 것은?

[T_EMP]

EMP_ID | MGR_ID | CODE | SALARY

1	10	A	100
2	NULL	B	300
3	NULL	C	150
4	20	B	250
5	NULL	B	180

[SQL]

```
SELECT COUNT(*)  
FROM T_EMP  
WHERE SALARY > 200  
OR MGR_ID IS NULL  
AND CODE = 'B';
```

- ① 1
- ② 2
- ③ 3
- ④ 4

정답: ③ 3

입문자용 상세 해설

논리 연산자는 괄호가 없을 때 AND 가 OR 보다 먼저 처리된다. 따라서 조건은 SALARY > 200 OR (MGR_ID IS NULL AND CODE = 'B')로 해석된다.

풀이 흐름

- EMP_ID 2: SALARY 300 이므로 SALARY > 200 이 TRUE 다. 조건 만족.
- EMP_ID 4: SALARY 250 이므로 SALARY > 200 이 TRUE 다. 조건 만족.
- EMP_ID 5: SALARY 는 180 이지만 MGR_ID IS NULL 이고 CODE=B 이므로 오른쪽 AND 조건이 TRUE 다. 조건 만족.
- EMP_ID 1 은 급여도 낮고 관리자도 NULL 이 아니므로 제외된다.
- EMP_ID 3 은 MGR_ID 는 NULL 이지만 CODE 가 C 라서 오른쪽 AND 조건이 FALSE 다.

흐름 도식

```
SALARY > 200 OR MGR_ID IS NULL AND CODE = 'B'  
= SALARY > 200 OR (MGR_ID IS NULL AND CODE = 'B')
```

비교 정리

우선순위	연산자	의미
1	NOT	부정
2	AND	둘 다 TRUE
3	OR	하나라도 TRUE

함정 포인트

원하는 해석이 다르면 괄호를 직접 써야 한다. 실무 SQL 에서도 조건이 복잡해지면 우선순위에 의존하지 말고 괄호를 쓰는 것이 안전하다.

문제 13. [집계 함수 - COUNT, DISTINCT] 다음 테이블에 대해 두 SQL 의 결과로 옳은 것은?

[TAB1]

COL1 | COL2

1 | A

2 | B

3 | C

4 | D

1 | A

2 | B

3 | A

[SQL1]

```
SELECT COUNT(ALL COL1) FROM TAB1 WHERE COL2 = 'A';
```

[SQL2]

```
SELECT COUNT(DISTINCT COL1) FROM TAB1;
```

① SQL1=2, SQL2=3

② SQL1=3, SQL2=3

③ SQL1=3, SQL2=4

④ SQL1=4, SQL2=4

정답: ③ SQL1=3, SQL2=4

입문자용 상세 해설

COUNT(ALL COL1)은 중복을 제거하지 않고 NULL 이 아닌 COL1 값을 센다. ALL 은 기본값이므로 생략해도 거의 같은 의미다.

COUNT(DISTINCT COL1)은 중복을 제거한 뒤 서로 다른 값의 개수를 센다.

풀이 흐름

- SQL1 은 먼저 COL2 = A 인 행만 남긴다. 해당 행의 COL1 은 1, 1, 3 이다.
- COUNT(ALL COL1)은 중복 1 을 그대로 세므로 3 건이다.
- SQL2 는 전체 COL1 값에서 중복을 제거한다. 서로 다른 값은 1, 2, 3, 4 이므로 4 개다.
-

비교 정리

함수/표현	NULL 처리	중복 처리	결과 의미
COUNT(*)	NULL 포함	중복 개념 없음	행 수
COUNT(컬럼)	NULL 제외	중복 포함	값이 있는 행 수
COUNT(ALL 컬럼)	NULL 제외	중복 포함	COUNT(컬럼)과 거의 동일
COUNT(DISTINCT 컬럼)	NULL 제외	중복 제거	서로 다른 값의 개수

함정 포인트

DISTINCT 가 붙은 경우에만 중복 제거가 일어난다. ALL 은 “모든 값을 그대로 센다”에 가깝다.

문제 14. [NULL 과 집계 함수] 다음 테이블과 SQL 의 결과로 옳은 것은?

[T_NULL]

COL1 | COL2

10 | 1

NULL | 2

30 | NULL

NULL | NULL

[SQL]

```
SELECT SUM(COL1) AS S1,  
       SUM(COL1 + COL2) AS S2,  
       COUNT(*) AS C1,  
       COUNT(COL1) AS C2  
FROM T_NULL;
```

- ① 40, 33, 4, 4
- ② 40, 11, 4, 2
- ③ NULL, 11, 4, 2
- ④ 40, NULL, 2, 2

정답: ② 40, 11, 4, 2

입문자용 상세 해설

집계 함수는 보통 NULL 을 무시하지만, 집계 전에 수행되는 산술식은 NULL 의 영향을 받는다. COL1 + COL2 는 두 값 중 하나라도 NULL 이면 결과가 NULL 이 된다.

풀이 흐름

- SUM(COL1): COL1 값 중 NULL 을 제외하고 10 + 30 = 40 이다.
- COL1 + COL2 는 행 단위로 먼저 계산한다. 첫 행 10+1=11 만 값이 있고, 나머지는 NULL 이 포함되어 계산 결과가 NULL 이다.
- SUM(COL1 + COL2)는 NULL 결과를 제외하고 11 만 더하므로 11 이다.
- COUNT(*)는 전체 행 수 4 를 센다.
- COUNT(COL1)은 COL1 이 NULL 이 아닌 행만 세므로 2 다.

비교 정리

행	COL1	COL2	COL1 + COL2
1	10	1	11
2	NULL	2	NULL
3	30	NULL	NULL
4	NULL	NULL	NULL

함정 포인트

“SUM 은 NULL 을 무시한다”와 “NULL 이 들어간 산술식은 NULL 이 된다”를 함께 기억해야 한다. SUM(COL1)과 SUM(COL1 + COL2)는 계산 순서가 다르다.

문제 15. [NULL 처리 함수] Oracle 기준으로 다음 SQL 의 결과로 옳은 것은?

[SQL]

```
SELECT NVL2(NULL, 100, 200) AS A,  
       NULLIF('SQL', 'SQL') AS B,  
       COALESCE(NULL, NULL, 'D', 'Q') AS C  
FROM DUAL;
```

- ① A=100, B=SQL, C=D
- ② A=200, B=NULL, C=D
- ③ A=NULL, B=NULL, C=Q
- ④ A=200, B=SQL, C=NULL

정답: ② A=200, B=NULL, C=D

입문자용 상세 해설

NULL 처리 함수는 “NULL 일 때 무엇을 반환할 것인가”를 다루는 함수다. 이 문제에서는 NVL2, NULLIF, COALESCE 의 반환 규칙을 각각 적용하면 A=200, B=NULL, C=D 가 된다.

풀이 흐름

- NVL2(NULL, 100, 200): 첫 번째 인수가 NULL 이면 세 번째 인수 200 을 반환한다.
- NULLIF('SQL', 'SQL'): 두 값이 같으면 NULL 을 반환한다.
- COALESCE(NULL, NULL, 'D', 'Q'): 왼쪽부터 보면서 처음으로 NULL 이 아닌 값 'D'를 반환한다.
-

비교 정리

함수	기능	이 문제에서의 결과	헛갈리는 점
NVL(expr, 대체값)	expr 이 NULL 이면 대체값, 아니면 expr	문제에는 직접 없음	인수가 2 개
NVL2(expr, 값 1, 값 2)	expr 이 NOT NULL 이면 값 1, NULL 이면 값 2	NVL2(NULL,100,200)=200	값 1/값 2 순서를 자주 바꿈
NULLIF(a,b)	a=b 이면 NULL, 다르면 a	NULLIF('SQL','SQL')=NULL	같으면 첫 값을 반환하는 함수가 아님
COALESCE(a,b,c,...)	왼쪽부터 첫 번째 NOT NULL 반환	COALESCE(NULL,NULL,'D','Q')='D'	여러 후보 중 첫 유효값 선택

함정 포인트

NVL2 는 “NULL 이면 두 번째”가 아니라 “NULL 이면 세 번째”다. 암기할 때 NVL2(expr, not_null_value, null_value) 순서로 기억하면 좋다.

문제 16. [문자 함수 - SUBSTR, INSTR] Oracle 기준으로 다음 중 실행 결과가 나머지와 다른 것은?

- ① SELECT SUBSTR('DATABASE', 7) FROM DUAL;
- ② SELECT SUBSTR('DATABASE', -2) FROM DUAL;
- ③ SELECT SUBSTR('DATABASE', 8, -2) FROM DUAL;
- ④ SELECT SUBSTR('DATABASE', INSTR('DATABASE', 'S'), 2) FROM DUAL;

정답: ③ SELECT SUBSTR('DATABASE', 8, -2) FROM DUAL;

입문자용 상세 해설

SUBSTR 은 문자열 일부를 잘라내고, INSTR 은 특정 문자의 위치를 찾는다. 'DATABASE'에서 7 번째 문자는 S, 8 번째 문자는 E 이므로 대부분 SE 가 나오지만, 길이 인수가 음수인 SUBSTR 은 Oracle 에서 NULL 을 반환한다.

풀이 흐름

- DATABASE 의 위치는 D=1, A=2, T=3, A=4, B=5, A=6, S=7, E=8 이다.
- SUBSTR('DATABASE', 7)은 7 번째 문자부터 끝까지 추출하므로 SE 다.
- SUBSTR('DATABASE', -2)는 뒤에서 두 번째 문자부터 끝까지 추출하므로 SE 다.
- SUBSTR('DATABASE', 8, -2)는 시작 위치는 8 이지만 길이 -2 가 유효하지 않으므로 NULL 이다.
- INSTR('DATABASE','S')는 S 의 위치 7 을 반환하고, SUBSTR(..., 7, 2)는 SE 를 반환한다.

비교 정리

함수	형식	기능	예시
SUBSTR	SUBSTR(문자열, 시작위치, 길이)	문자열 일부 추출	SUBSTR('DATABASE',7) = 'SE'
INSTR	INSTR(문자열, 찾을문자)	문자의 위치 반환	INSTR('DATABASE','S') = 7
LENGTH	LENGTH(문자열)	문자열 길이 반환	LENGTH('DATABASE') = 8

함정 포인트

시작 위치가 음수인 것은 “뒤에서부터 위치를 센다”는 의미라 가능하지만, 길이가 음수인 것은 추출할 길이가 없다는 뜻이므로 NULL 이 된다.

문제 17. [LIKE, 와일드카드, ESCAPE] 다음 데이터에서 실제 언더바(_) 문자가 포함된 USERNAME 만 조회하려고 한다. 가장 적절한 SQL 은?

[USERS]

USER_ID | USERNAME

1 | AB_C
2 | ABC
3 | A_C
4 | A#C
5 | XYZ_

- ① SELECT * FROM USERS WHERE USERNAME LIKE '%_%';
- ② SELECT * FROM USERS WHERE USERNAME LIKE '%#_%';
- ③ SELECT * FROM USERS WHERE USERNAME LIKE '%!_%' ESCAPE '!';
- ④ SELECT * FROM USERS WHERE USERNAME LIKE '%_%' ESCAPE '_';

정답: ③ SELECT * FROM USERS WHERE USERNAME LIKE '%!_%' ESCAPE '!';

입문자용 상세 해설

LIKE 에서 _는 실제 언더바 문자가 아니라 “임의의 한 문자”를 의미하는 와일드카드다. 실제 _ 문자를 찾으려면 ESCAPE 문자를 지정해서 _를 일반 문자로 해석하게 만들어야 한다.

풀이 흐름

- LIKE '%_%'는 언더바가 포함된 문자열이 아니라, 길이가 1 자 이상인 대부분의 문자열을 찾는다.
- LIKE '%!_%' ESCAPE '!'에서 !는 escape 문자다.
- 따라서 !_는 와일드카드가 아니라 실제 언더바 문자 _를 의미한다.
- 결과적으로 AB_C, A_C, XYZ_처럼 실제 _가 들어 있는 행을 찾을 수 있다.

비교 정리

패턴 요소	의미	예시
%	0 글자 이상 아무 문자열	'A%'는 A 로 시작하는 값
-	정확히 1 글자 아무 문자	'A_C'는 ABC, A1C 도 가능
ESCAPE	와일드카드를 일반 문자로 해석하게 함	LIKE '%!_%' ESCAPE '!'
!_	실제 언더바 문자	AB_C, XYZ_의 _

함정 포인트

ESCAPE 문자는 데이터에서 찾고 싶은 문자가 아니라 “다음 문자를 특별한 의미가 아닌 일반 문자로 보라”는 표시다.

문제 18. [정규표현식 - REGEXP_INSTR] Oracle 기준으로 다음 SQL 의 결과로 옳은 것은?

[SQL]

```
SELECT REGEXP_INSTR('ABC123XYZ',  
    '([A-Z]+)([0-9]+)([A-Z]+)',  
    1, 1, 0, 'c', 2) AS POS  
FROM DUAL;
```

- ① 1
- ② 4
- ③ 7
- ④ 10

정답: ② 4

입문자용 상세 해설

REGEXP_INSTR 은 정규표현식으로 찾은 패턴의 위치를 반환한다. 마지막 인수 subexpr 이 2 이므로 전체 패턴의 시작 위치가 아니라 두 번째 괄호 그룹의 시작 위치를 반환한다.

풀이 흐름

- 패턴 ([A-Z]+)([0-9]+)([A-Z]+)은 세 그룹으로 나뉜다.
- 문자열 ABC123XYZ 에서 첫 번째 그룹 ([A-Z]+)은 ABC 이고 시작 위치는 1 이다.
- 두 번째 그룹 ([0-9]+)은 123 이고 시작 위치는 4 이다.
- 세 번째 그룹 ([A-Z]+)은 XYZ 이고 시작 위치는 7 이다.
- subexpr=2 이므로 두 번째 그룹 123 의 시작 위치 4 가 결과다.

비교 정리

정규표현식 함수	반환값	주요 용도
REGEXP_LIKE	TRUE/FALSE 조건	패턴과 일치하는 행 검색
REGEXP_INSTR	일치 위치	패턴이 몇 번째 위치에서 시작/끝나는지 확인
REGEXP_SUBSTR	일치 문자열	패턴에 맞는 부분 문자열 추출
REGEXP_REPLACE	치환된 문자열	패턴에 맞는 부분을 다른 값으로 변경

함정 포인트

정규표현식에서 괄호는 단순 묶음이면서 동시에 “캡처 그룹”이 될 수 있다. REGEXP_INSTR 의 subexpr 인수는 이 그룹 번호를 지정한다.

문제 19. [정규표현식 - REGEXP_SUBSTR] Oracle 기준으로 다음 SQL 의 결과로 옳은 것은?

[SQL]

```
SELECT REGEXP_SUBSTR('ccaaaabb', 'a{2,3}', 1, 1) AS R
FROM DUAL;
```

- ① aa
- ② aaa
- ③ aaaa
- ④ NULL

정답: ② aaa

입문자용 상세 해설

REGEXP_SUBSTR 은 정규표현식에 맞는 문자열 자체를 반환한다. a{2,3}은 a 가 최소 2 번, 최대 3 번 반복되는 부분을 의미한다.

풀이 흐름

- 문자열 ccaaaabb 에서 a 가 연속되는 부분은 aaaa 다.
- 패턴 a{2,3}은 a 2 개 또는 3 개까지만 허용한다.
- Oracle 정규표현식은 가능한 범위 안에서 길게 매칭하려는 탐욕적 방식이므로 aaa 를 선택한다.
- 최대 반복 수가 3 이므로 aaaa 전체는 반환될 수 없다.
-

비교 정리

표현식	의미	예시
a{2}	a 가 정확히 2 회	aa
a{2,3}	a 가 2 회 이상 3 회 이하	aa 또는 aaa
a+	a 가 1 회 이상	a, aa, aaa 등
a*	a 가 0 회 이상	빈 문자열도 가능

함정 포인트

문자열에 a 가 네 개 있어도 패턴이 허용하는 최대 반복 수는 3 이다. “데이터에 있는 길이”가 아니라 “패턴이 허용하는 길이”를 봐야 한다.

문제 20. [SQL 명령어 분류] SQL 명령어 분류에 대한 설명으로 옳지 않은 것은?

- ① TRUNCATE 는 DDL 로 분류할 수 있다.
- ② DROP 은 DML 로 분류한다.
- ③ REVOKE 는 DCL 로 분류한다.
- ④ COMMIT 은 TCL 로 분류한다.

정답: ② DROP 은 DML 로 분류한다.

입문자용 상세 해설

DROP 은 테이블, 뷰, 인덱스 같은 데이터베이스 객체 자체를 삭제하는 DDL 이다. DML 은 데이터 행을 조회하거나 조작하는 명령이고, DROP 처럼 객체 구조를 제거하는 명령은 DML 이 아니다.

풀이 흐름

- TRUNCATE 는 테이블의 모든 행을 빠르게 제거하지만 테이블 구조를 대상으로 하는 DDL 로 분류한다.
- DROP 은 객체 자체를 삭제하므로 DDL 이다.
- REVOKE 는 권한을 회수하므로 DCL 이다.
- COMMIT 은 트랜잭션을 확정하므로 TCL 이다.

비교 정리

분류	대표 명령어	기능
DDL	CREATE, ALTER, DROP, TRUNCATE	객체 생성/변경/삭제
DML	SELECT, INSERT, UPDATE, DELETE	데이터 조회/입력/수정/삭제. 일부 교재는 SELECT 를 DQL 로 별도 분류
DCL	GRANT, REVOKE	권한 부여/회수
TCL	COMMIT, ROLLBACK, SAVEPOINT	트랜잭션 제어

함정 포인트

DELETE 와 DROP 은 둘 다 “삭제” 느낌이 있지만 대상이 다르다. DELETE 는 테이블 안의 행 삭제, DROP 은 테이블 같은 객체 자체 삭제다.

문제 21. [DDL 과 자동 COMMIT] Oracle 기준으로 다음 스크립트 수행 후 SELECT 결과로 옳은 것은?

[SQL]

```
CREATE TABLE T_DDL (COL1 NUMBER);
INSERT INTO T_DDL VALUES (1);
SAVEPOINT S1;
INSERT INTO T_DDL VALUES (2);
CREATE TABLE T_DDL_LOG (COL1 NUMBER);
```

```
ROLLBACK;
SELECT COUNT(*) FROM T_DDL;
```

- ① 0
- ② 1
- ③ 2

④ 오류가 발생한다.

정답: ③ 2

입문자용 상세 해설

Oracle 에서 DDL 은 트랜잭션을 자동으로 확정시키는 효과가 있다. CREATE TABLE T_DDL_LOG 가 실행되는 순간 그 전에 수행한 INSERT 1, INSERT 2 가 커밋되므로 이후 ROLLBACK 으로 되돌릴 수 없다.

풀이 흐름

- CREATE TABLE T_DDL 은 DDL 이다. 테이블이 생성된다.
- INSERT 1, SAVEPOINT S1, INSERT 2 가 수행되어 T_DDL 에는 1 과 2 가 들어간 상태다.
- CREATE TABLE T_DDL_LOG 가 수행되면서 앞선 DML 변경분이 커밋된다.
- ROLLBACK 을 실행해도 이미 커밋된 1, 2 는 취소되지 않는다.
- 따라서 SELECT COUNT(*) 결과는 2 다.

흐름 도식

```
INSERT 1 → SAVEPOINT → INSERT 2 → DDL 수행
      ↓
    자동 COMMIT
      ↓
    ROLLBACK 불가
```

비교 정리

상황	ROLLBACK 가능 여부
아직 COMMIT 전의 DML	가능
COMMIT 된 DML	불가능
Oracle 에서 DDL 수행 전후로 확정된 변경	불가능

함정 포인트

SAVEPOINT 는 같은 트랜잭션 안에서만 의미가 있다. DDL 로 트랜잭션 경계가 끊기면 이전 SAVEPOINT 로 돌아갈 수 없다.

문제 22. [TCL - SAVEPOINT, ROLLBACK] Oracle 기준으로 다음 SQL 수행 후 결과로 옳은 것은?

[SQL]

```
CREATE TABLE T_SP (C NUMBER);
INSERT INTO T_SP VALUES (2);
INSERT INTO T_SP VALUES (7);
SAVEPOINT P;
UPDATE T_SP SET C = 9 WHERE C = 1;
SAVEPOINT P;
DELETE FROM T_SP WHERE C >= 7;
ROLLBACK TO P;
INSERT INTO T_SP VALUES (5);
SELECT COUNT(*) AS CNT, MAX(C) AS MX FROM T_SP;
```

- ① CNT=2, MX=5
- ② CNT=2, MX=7
- ③ CNT=3, MX=7
- ④ CNT=3, MX=9

정답: ③ CNT=3, MX=7

입문자용 상세 해설

SAVEPOINT 는 트랜잭션 중간 저장점이다. 같은 이름의 SAVEPOINT 를 다시 만들면 마지막 저장점이 기준이 된다. 이 문제에서는 두 번째 SAVEPOINT P 가 DELETE 직전 상태를 가리킨다.

풀이 흐름

- 처음에는 2 와 7 이 INSERT 된다.
- 첫 번째 SAVEPOINT P 를 만든다.
- UPDATE T_SP SET C=9 WHERE C=1 은 C=1 인 행이 없으므로 아무 변화가 없다.
- 두 번째 SAVEPOINT P 를 만든다. 이제 P 는 이 시점을 가리킨다.
- DELETE FROM T_SP WHERE C >= 7 로 7 이 삭제된다.
- ROLLBACK TO P 로 DELETE 직전으로 돌아가므로 7 이 복구된다.
- INSERT 5 를 수행하므로 최종 데이터는 2, 7, 5 다. COUNT=3, MAX=7 이다.

흐름 도식

2,7 삽입 → P 저장 → 변화 없는 UPDATE → P 재저장 → 7 삭제 → ROLLBACK TO P → 5 삽입
최종: 2, 7, 5

비교 정리

명령	데이터 상태
INSERT 2, INSERT 7	2, 7
UPDATE C=1	변화 없음: 2, 7
DELETE C>=7	2
ROLLBACK TO P	2, 7
INSERT 5	2, 7, 5

함정 포인트

SAVEPOINT 이름이 중복되면 “처음 P”가 아니라 “가장 최근 P”가 기준이 된다.

문제 23. [MERGE - UPDATE 와 DELETE] Oracle 기준으로 다음 MERGE 수행 후 T_M1 의 최종 행 수로 옳은 것은?

[T_M1]
ID | VAL
A | 1
B | 2
C | 3

[T_M2]
 ID | VAL
 A | 1
 B | 2
 C | 3
 D | 4
 [SQL]

```

MERGE INTO T_M1 T
USING T_M2 S
  ON (T.ID = S.ID)
WHEN MATCHED THEN
  UPDATE SET T.VAL = 5
  WHERE T.VAL = 2
  DELETE WHERE T.VAL >= 5
WHEN NOT MATCHED THEN
  INSERT (ID, VAL) VALUES (S.ID, S.VAL);

```

- ① 2
- ② 3
- ③ 4
- ④ 5

정답: ② 3

입문자용 상세 해설

MERGE 는 대상 테이블과 원본 테이블을 비교해 매칭되면 UPDATE/DELETE, 매칭되지 않으면 INSERT 를 수행하는 문장이다. 이 문제에서는 B 만 업데이트된 뒤 삭제되고, D 는 새로 삽입된다.

풀이 흐름

- A, B, C 는 T_M1 과 T_M2 양쪽에 있으므로 MATCHED 다. D 는 T_M1 에 없으므로 NOT MATCHED 다.
- UPDATE 의 WHERE 조건은 T.VAL = 2 이므로 B 만 VAL=5 로 변경된다.
- DELETE WHERE T.VAL >= 5 는 MERGE 에서 UPDATE 된 행을 대상으로, 업데이트 후 값을 기준으로 평가된다.
- B 는 업데이트 후 VAL=5 이므로 삭제된다.
- D 는 NOT MATCHED 이므로 INSERT 된다.
- 최종 T_M1 에는 A, C, D 가 남아 총 3 건이다.

비교 정리

ID	처리 구분	UPDATE 여부	DELETE 여부	최종 상태
A	MATCHED	조건 불만족	대상 아님	유지
B	MATCHED	VAL 2→5	VAL>=5 로 삭제	삭제
C	MATCHED	조건 불만족	대상 아님	유지
D	NOT MATCHED	-	-	삽입

함정 포인트

MERGE 의 DELETE WHERE 를 MATCHED 전체 행에 적용한다고 생각하면 오답이 된다. 여기서는 UPDATE 가 수행된 행의 업데이트 후 값이 핵심이다.

문제 24. [DEFAULT 와 NULL 입력] Oracle 기준으로 다음 SQL 수행 후 STATUS='N'인 행 수와 STATUS IS NULL 인 행 수로 옳은 것은?

```
[SQL]
CREATE TABLE T_DEF (
  ID NUMBER,
  STATUS VARCHAR2(1) DEFAULT 'N'
);
INSERT INTO T_DEF (ID) VALUES (1);
INSERT INTO T_DEF (ID, STATUS) VALUES (2, NULL);
INSERT INTO T_DEF (ID, STATUS) VALUES (3, DEFAULT);
```

- ① STATUS='N': 1 건, NULL: 2 건
- ② STATUS='N': 2 건, NULL: 1 건
- ③ STATUS='N': 3 건, NULL: 0 건
- ④ STATUS='N': 0 건, NULL: 3 건

정답: ② STATUS='N': 2 건, NULL: 1 건

입문자용 상세 해설

DEFAULT 는 컬럼을 생략하거나 DEFAULT 키워드를 명시했을 때 기본값을 넣는다. 하지만 컬럼에 NULL 을 직접 넣으면 일반 DEFAULT 는 그 NULL 을 기본값으로 바꾸지 않는다.

풀이 흐름

- INSERT INTO T_DEF (ID) VALUES (1): STATUS 컬럼을 생략했으므로 DEFAULT N 이 들어간다.
- INSERT INTO T_DEF (ID, STATUS) VALUES (2, NULL): STATUS 에 NULL 을 직접 넣었으므로 NULL 이 저장된다.
- INSERT INTO T_DEF (ID, STATUS) VALUES (3, DEFAULT): DEFAULT 키워드를 썼으므로 N 이 들어간다.
- 따라서 STATUS='N'은 2 건, STATUS IS NULL 은 1 건이다.

비교 정리

입력 방식	저장되는 STATUS
컬럼 생략	기본값 N
명시적 NULL 입력	NULL
DEFAULT 키워드 입력	기본값 N

함정 포인트

Oracle 에는 DEFAULT ON NULL 같은 별도 기능도 있지만, 이 문제의 컬럼 정의는 단순 DEFAULT 다. 단순 DEFAULT 는 명시적 NULL 을 자동 변환하지 않는다.

문제 25. [제약조건 - FK, NOT NULL, SET NULL] 다음 제약조건이 설정되어 있을 때, 부모 테이블 PARENT 의 ID=10 인 행을 삭제하려고 한다. CHILD 에 P_ID=10 인 행이 존재한다면 결과로 가장 적절한 것은?

[조건]

- CHILD.P_ID 는 PARENT.ID 를 참조하는 외래키다.
 - 외래키 옵션은 ON DELETE SET NULL 이다.
 - CHILD.P_ID 에는 NOT NULL 제약조건이 있다.
- ① 부모 행만 삭제되고 자식 행은 유지된다.
② 자식 행의 P_ID 가 NULL 로 바뀐 뒤 부모 행이 삭제된다.
③ NOT NULL 제약조건 때문에 삭제가 실패한다.
④ 자식 행이 자동 삭제된다.

정답: ③ NOT NULL 제약조건 때문에 삭제가 실패한다.

입문자용 상세 해설

ON DELETE SET NULL 은 부모 행이 삭제될 때 자식 테이블의 외래키 값을 NULL 로 바꾸는 옵션이다. 그런데 자식 외래키 컬럼에 NOT NULL 제약이 있으면 NULL 로 바꿀 수 없으므로 부모 삭제가 실패한다.

풀이 흐름

- 부모 PARENT.ID=10 을 삭제하려고 한다.
- CHILD 에 P_ID=10 인 자식 행이 존재한다.
- 외래키 옵션 ON DELETE SET NULL 때문에 DBMS 는 CHILD.P_ID 를 NULL 로 바꾸려고 한다.
- 하지만 CHILD.P_ID 는 NOT NULL 이므로 NULL 저장이 불가능하다.
- 결과적으로 부모 행 삭제가 실패한다.

비교 정리

외래키 삭제 옵션	부모 삭제 시 자식 처리
ON DELETE CASCADE	자식 행도 함께 삭제
ON DELETE SET NULL	자식 FK 값을 NULL 로 변경
옵션 없음/제한	자식이 있으면 부모 삭제 제한

함정 포인트

SET NULL 은 “자식 삭제”가 아니다. 자식 행은 남기고 FK 만 NULL 로 바꾸는 방식이므로, FK 컬럼이 NOT NULL 이면 충돌한다.

문제 26. [표준 조인 - 조인 결과 행 수] 다음 두 테이블을 KEY 컬럼으로 조인할 때 INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN 결과 행 수의 합으로 옳은 것은?

- [A]
KEY
1
2
3
- [B]
KEY
2
3
4
- ① 10
② 11
③ 12
④ 13

정답: ③ 12

입문자용 상세 해설

두 테이블 A 와 B 를 KEY 로 조인할 때, INNER JOIN 은 양쪽에 모두 있는 키만 남기고, OUTER JOIN 은 기존 테이블의 미매칭 행도 보존한다. 이 문제는 중복 키가 없으므로 키 집합으로 쉽게 계산할 수 있다.

풀이 흐름

- A 의 키는 1, 2, 3 이고 B 의 키는 2, 3, 4 다.
- INNER JOIN: 공통 키 2, 3 이므로 2 건이다.
- LEFT OUTER JOIN: A 의 모든 키 1, 2, 3 이 보존되므로 3 건이다.
- RIGHT OUTER JOIN: B 의 모든 키 2, 3, 4 가 보존되므로 3 건이다.
- FULL OUTER JOIN: 양쪽 키의 합집합 1, 2, 3, 4 가 보존되므로 4 건이다.
- 합계는 2+3+3+4=12 다.

비교 정리

조인 종류	남는 키	행 수
INNER JOIN	2, 3	2
LEFT OUTER JOIN	1, 2, 3	3
RIGHT OUTER JOIN	2, 3, 4	3
FULL OUTER JOIN	1, 2, 3, 4	4

함정 포인트

FULL OUTER JOIN 은 왼쪽 행 수 + 오른쪽 행 수가 아니다. 공통 키는 한 번의 매칭 결과로 나타난다. 단, 실제 문제에서 중복 키가 있으면 매칭 행 수가 곱으로 늘 수 있다.

문제 27. [NATURAL JOIN, USING] Oracle 기준으로 EMP(EMPNO, ENAME, DEPTNO)와 DEPT(DEPTNO, DNAME)가 있을 때 NATURAL JOIN 에 대한 설명으로 옳지 않은 것은?

- ① NATURAL JOIN 은 이름이 같은 컬럼을 자동으로 조인 조건에 사용한다.
- ② NATURAL JOIN 결과에서 조인에 사용된 공통 컬럼은 한 번만 출력된다.
- ③ NATURAL JOIN 에서 사용된 공통 컬럼을 SELECT 절에서 E.DEPTNO 처럼 테이블 별칭으로 한정할 수 있다.
- ④ 명시적 조인 조건을 쓰고 싶다면 INNER JOIN ... ON 구문을 사용할 수 있다.

정답: ③ NATURAL JOIN 에서 사용된 공통 컬럼을 SELECT 절에서 E.DEPTNO 처럼 테이블 별칭으로 한정할 수 있다.

입문자용 상세 해설

NATURAL JOIN 은 양쪽 테이블에서 이름이 같은 컬럼을 자동으로 찾아 조인 조건으로 사용한다. Oracle 에서는 NATURAL JOIN 이나 USING 에 사용된 공통 컬럼을 SELECT 에서 E.DEPTNO 처럼 별칭으로 한정할 수 없다.

풀이 흐름

- EMP 와 DEPT 에는 DEPTNO 라는 같은 이름의 컬럼이 있다.
- NATURAL JOIN 은 이 DEPTNO 를 자동 조인 조건으로 사용한다.
- 조인 결과에서 공통 컬럼 DEPTNO 는 중복 출력되지 않고 한 번만 나타난다.
- Oracle 에서는 이 공통 컬럼을 E.DEPTNO 처럼 접두사로 지정하면 오류가 날 수 있으므로 DEPTNO 처럼 컬럼명만 사용해야 한다.

비교 정리

조인 문법	조인 조건 지정 방식	장점/주의점
NATURAL JOIN	같은 이름 컬럼을 자동 사용	간단하지만 같은 이름 컬럼 추가 시 결과가 바뀔 수 있음
JOIN ... USING(컬럼)	공통 컬럼명을 명시	공통 컬럼은 결과에서 한 번만 출력
JOIN ... ON	별칭.컬럼 = 별칭.컬럼으로 직접 지정	가장 명확하고 실무에서 선호됨

함정 포인트

일반 조인에서는 별칭.컬럼 형식이 안전하지만, NATURAL JOIN/USING 의 공통 컬럼에서는 오히려 오류 포인트가 될 수 있다.

문제 28. [CROSS JOIN] A 테이블에 4 건, B 테이블에 3 건의 행이 있다. 다음 SQL 의 COUNT(*) 결과로 옳은 것은?

```
[SQL]
SELECT COUNT(*)
FROM A CROSS JOIN B;
```

- ① 3
- ② 4
- ③ 7
- ④ 12

정답: ④ 12

입문자용 상세 해설

CROSS JOIN 은 조인 조건 없이 양쪽 테이블의 모든 조합을 만든다. A 에 4 행, B 에 3 행이 있으면 가능한 조합은 4×3=12 행이다.

풀이 흐름

- A 의 첫 번째 행은 B 의 3 개 행과 각각 결합된다.
- A 의 두 번째 행도 B 의 3 개 행과 각각 결합된다.
- 이 과정이 A 의 4 개 행에 대해 반복되므로 전체 조합 수는 4×3 이다.
-

흐름 도식

A 4 행 × B 3 행 = 12 개 조합

비교 정리

조인 종류	조건 필요 여부	결과 특징
INNER JOIN	필요	조건에 맞는 행만 결합
OUTER JOIN	필요	미매칭 행도 보존 가능
CROSS JOIN	필요 없음	모든 경우의 수를 생성

함정 포인트

CROSS JOIN 은 누락된 행을 보존하는 OUTER JOIN 이 아니다. 경우의 수를 만들기 때문에 데이터가 많으면 결과 행 수가 폭발적으로 늘 수 있다.

문제 29. [OUTER JOIN - 조건 위치] 모든 고객을 출력하되, 상태가 'Y'인 주문 금액만 함께 조회하려고 한다. 주문이 없거나 상태가 'Y'인 주문이 없는 고객도 결과에 남아야 한다. 가장 적절한 SQL 은?

[CUSTOMER]
CID | CNAME
1 | A
2 | B
3 | C

[ORDERS]
CID | STATUS | AMT
1 | Y | 100
1 | N | 200
2 | N | 300

- ① SELECT C.CID, O.AMT FROM CUSTOMER C INNER JOIN ORDERS O ON C.CID=O.CID AND O.STATUS='Y';
 - ② SELECT C.CID, O.AMT FROM CUSTOMER C LEFT OUTER JOIN ORDERS O ON C.CID=O.CID WHERE O.STATUS='Y';
 - ③ SELECT C.CID, O.AMT FROM CUSTOMER C LEFT OUTER JOIN ORDERS O ON C.CID=O.CID AND O.STATUS='Y';
 - ④ SELECT C.CID, O.AMT FROM CUSTOMER C RIGHT OUTER JOIN ORDERS O ON C.CID=O.CID WHERE O.STATUS='Y';
- 정답: ③ SELECT C.CID, O.AMT FROM CUSTOMER C LEFT OUTER JOIN ORDERS O ON C.CID=O.CID AND O.STATUS='Y';

입문자용 상세 해설

모든 고객을 결과에 남겨야 하므로 CUSTOMER 를 기준으로 LEFT OUTER JOIN 을 사용해야 한다. 그리고 오른쪽 테이블 ORDERS 의 상태 조건은 WHERE 가 아니라 ON 절에 두어야 고객 보존 효과가 유지된다.

풀이 흐름

- 고객 1 은 상태 Y 주문이 있으므로 AMT=100 이 매칭된다.
- 고객 2 는 주문은 있지만 상태가 N 뿐이다. 상태 Y 조건을 ON 절에 두면 매칭 주문이 없는 것으로 처리되어 고객 2 는 NULL 금액으로 남는다.
- 고객 3 은 주문이 없으므로 역시 NULL 금액으로 남는다.
- 만약 WHERE O.STATUS='Y'를 쓰면 조인 후 NULL 확장된 행이 WHERE 에서 제거되어 고객 2, 3 이 사라진다.

흐름 도식

LEFT JOIN 의 ON 조건: 매칭 대상을 제한하지만 왼쪽 고객은 보존

LEFT JOIN 후 WHERE 조건: 최종 결과를 필터링하므로 NULL 행이 제거될 수 있음

비교 정리

조건 위치	의미	모든 고객 보존 여부
ON 절	조인할 오른쪽 행을 제한	보존 가능
WHERE 절	조인 결과 전체를 필터링	오른쪽 NULL 행이 제거될 수 있음

함정 포인트

OUTER JOIN 에서 오른쪽 테이블 조건을 WHERE 에 두면 INNER JOIN 처럼 변하는 경우가 많다. “기준 테이블을 반드시 남겨야 하는가”를 먼저 확인해야 한다.

문제 30. [서브쿼리 - IN 과 EXISTS] 다음 SQL 과 동일한 결과를 반환하는 SQL 로 가장 적절한 것은?

[SQL]

```
SELECT *  
FROM A  
WHERE (ID, SEX) IN (  
    SELECT ID, SEX  
    FROM B
```

);

- ① SELECT * FROM A WHERE EXISTS (SELECT 1 FROM B WHERE B.ID = A.ID);
- ② SELECT * FROM A WHERE EXISTS (SELECT 1 FROM B WHERE B.SEX = A.SEX);
- ③ SELECT * FROM A WHERE EXISTS (SELECT 1 FROM B WHERE B.ID = A.ID AND B.SEX = A.SEX);
- ④ SELECT * FROM A WHERE NOT EXISTS (SELECT 1 FROM B WHERE B.ID = A.ID AND B.SEX = A.SEX);

정답: ③ SELECT * FROM A WHERE EXISTS (SELECT 1 FROM B WHERE B.ID = A.ID AND B.SEX = A.SEX);

입문자용 상세 해설

다중 컬럼 IN 은 여러 컬럼의 조합이 같은지를 비교한다. (ID, SEX) IN (...)은 ID 만 맞거나 SEX 만 맞으면 되는 조건이 아니라, ID 와 SEX 가 같은 행 조합이 B 에 존재해야 한다.

풀이 흐름

- A 의 한 행을 기준으로 B 에 같은 ID 와 같은 SEX 를 가진 행이 있는지 확인한다.
- EXISTS 로 바꿀 때도 같은 조합 조건을 유지해야 한다.
- 따라서 B.ID = A.ID AND B.SEX = A.SEX 두 조건이 모두 필요하다.
- ID 만 비교하거나 SEX 만 비교하면 원래 SQL 보다 더 많은 행이 반환될 수 있다.

비교 정리

표현	의미	주의점
IN	값이 목록 또는 서브쿼리 결과 안에 있는지 확인	단일 컬럼 또는 다중 컬럼 비교 가능
EXISTS	서브쿼리 결과 행이 하나라도 존재하는지 확인	상관 조건을 정확히 작성해야 함
다중 컬럼 IN	컬럼 조합 단위로 비교	(ID, SEX) 전체가 일치해야 함

함정 포인트

EXISTS 의 SELECT 1 에서 1 은 실제 반환값이 중요하다는 뜻이 아니다. EXISTS 는 “행 존재 여부”만 본다.

문제 31. [서브쿼리 - NOT IN 과 NULL] 다음 SQL 의 결과 행 수로 옳은 것은?

[A]

ID

1

2

3

[B]

ID

2

NULL

[SQL]

```
SELECT COUNT(*)
FROM A
WHERE ID NOT IN (SELECT ID FROM B);
```

① 0

② 1

③ 2

④ 3

정답: ① 0

입문자용 상세 해설

NOT IN 의 하위 질의 결과에 NULL 이 포함되면 조건 결과가 UNKNOWN 이 되어 기대와 다르게 모든 행이 제외될 수 있다. 이 문제의 B 에는 2 와 NULL 이 있으므로 A 의 1, 2, 3 모두 WHERE 조건을 TRUE 로 만들지 못한다.

풀이 흐름

- ID NOT IN (2, NULL)은 논리적으로 ID <> 2 AND ID <> NULL 과 비슷하게 해석된다.
- ID <> NULL 의 결과는 TRUE 나 FALSE 가 아니라 UNKNOWN 이다.
- WHERE 절은 TRUE 인 행만 통과시키고 FALSE 나 UNKNOWN 은 제외한다.
- 따라서 1 과 3 도 NULL 비교 때문에 UNKNOWN 이 섞여 통과하지 못한다.
- 결과 행 수는 0 이다.

비교 정리

A.ID	ID<>2	ID<>NULL	최종 판단
1	TRUE	UNKNOWN	UNKNOWN → 제외
2	FALSE	UNKNOWN	FALSE → 제외
3	TRUE	UNKNOWN	UNKNOWN → 제외

함정 포인트

NULL 가능성이 있는 컬럼에 NOT IN 을 사용할 때는 매우 조심해야 한다. 실무에서는 NOT EXISTS 또는 서브쿼리에서 WHERE ID IS NOT NULL 을 함께 검토한다.

문제 32. [서브쿼리 - 스칼라 서브쿼리] 다음 SQL 수행 결과로 가장 적절한 것은?

[DEPT]

DEPTNO

10

20

[EMP]

EMPNO | ENAME | DEPTNO

1 | A | 10

2 | B | 10

3 | C | 20

[SQL]

```
SELECT D.DEPTNO,
       (SELECT E.ENAME
        FROM EMP E
        WHERE E.DEPTNO = D.DEPTNO) AS ENAME
FROM DEPT D;
```

- ① DEPTNO 10 에는 A 만 출력되고 정상 수행된다.
- ② DEPTNO 10 에는 A 와 B 가 한 컬럼에 연결되어 출력된다.
- ③ 스칼라 서브쿼리가 2 행 이상을 반환할 수 있어 오류가 발생한다.
- ④ 서브쿼리는 SELECT 절에서 사용할 수 없으므로 항상 오류다.

정답: ③ 스칼라 서브쿼리가 2 행 이상을 반환할 수 있어 오류가 발생한다.

입문자용 상세 해설

스칼라 서브쿼리는 “값 하나”를 반환해야 한다. SELECT 절의 한 칸에 들어가는 값이므로 한 행, 한 컬럼만 가능하다. 그런데 DEPTNO=10 인 사원은 A 와 B 두 명이므로 서브쿼리가 2 행을 반환할 수 있어 오류가 발생한다.

풀이 흐름

- DEPT 테이블에서 DEPTNO=10 행을 처리한다.
- 서브쿼리 SELECT E.ENAME FROM EMP WHERE E.DEPTNO=10 은 A 와 B 두 행을 반환한다.
- SELECT 목록의 ENAME 한 칸에는 값 하나만 들어가야 하므로 DBMS 가 어떤 값을 넣을지 결정할 수 없다.
- 따라서 단일 행 서브쿼리가 여러 행을 반환했다는 오류가 발생한다.

비교 정리

서브쿼리 종류	반환 형태	사용 예
스칼라 서브쿼리	1 행 1 컬럼	SELECT 절의 계산 컬럼
단일행 서브쿼리	1 행	=, >, < 비교
다중행 서브쿼리	여러 행	IN, EXISTS, ANY, ALL
인라인 뷰	테이블 형태	FROM 절의 서브쿼리

함정 포인트

해결하려면 MAX(E.ENAME)처럼 집계로 하나를 선택하거나, 조인으로 여러 사원을 여러 행으로 출력하는 방식으로 바꿔야 한다.

문제 33. [다중 컬럼 서브쿼리와 NULL] 다음 SQL 의 결과 행 수로 옳은 것은?

[T1]

A | B

1 | 10

1 | 20

2 | 10

3 | NULL

[T2]

A | B

1 | 10

2 | NULL

[SQL]

```
SELECT COUNT(*)
```

```
FROM T1
```

```
WHERE (A, B) IN (SELECT A, B FROM T2);
```

① 0

② 1

③ 2

④ 3

정답: ② 1

입문자용 상세 해설

다중 컬럼 IN 에서도 NULL 은 일반 값처럼 = 비교로 같다고 판단되지 않는다. T1 의 (1,10)만 T2 의 (1,10)과 정확히 일치하므로 결과는 1 건이다.

풀이 흐름

- (1,10)은 T2 에 같은 조합 (1,10)이 있으므로 일치한다.
- (1,20)은 T2 의 (1,10)과 B 가 다르므로 불일치한다.
- (2,10)은 T2 에 (2,NULL)이 있지만 10 = NULL 은 TRUE 가 아니므로 일치하지 않는다.
- (3,NULL)은 T2 에 A=3 이 없고, NULL 도 = 비교로 같다고 볼 수 없으므로 일치하지 않는다.

비교 정리

T1 행	T2 와 비교 결과	포함 여부
(1,10)	(1,10)과 일치	포함
(1,20)	B 값 불일치	제외
(2,10)	10 = NULL 은 UNKNOWN	제외
(3,NULL)	일치하는 A 도 없고 NULL 비교도 TRUE 아님	제외

함정 포인트

NULL 끼리도 = 연산으로는 같다고 판단되지 않는다. NULL 여부를 비교하려면 IS NULL 을 사용해야 한다.

문제 34. [GROUP BY, HAVING] 다음 SQL 이 오류가 나는 이유로 가장 적절한 것은?

[SQL]

```
SELECT DEPTNO, ENAME, SUM(SAL)
FROM EMP
GROUP BY DEPTNO;
```

- ① SUM 함수는 SELECT 절에서 사용할 수 없다.
- ② GROUP BY 절 이후에는 그룹 기준 컬럼과 집계 함수 결과 외의 일반 컬럼을 SELECT 할 수 없다.
- ③ GROUP BY 절에는 숫자 컬럼만 사용할 수 있다.
- ④ DEPTNO 는 반드시 WHERE 절에 있어야 한다.

정답: ② GROUP BY 절 이후에는 그룹 기준 컬럼과 집계 함수 결과 외의 일반 컬럼을 SELECT 할 수 없다.

입문자용 상세 해설

GROUP BY 를 사용하면 SELECT 절에는 그룹 기준 컬럼과 집계 함수 결과만 직접 쓸 수 있다. DEPTNO 로 그룹화하면 한 부서에 여러 ENAME 이 있을 수 있으므로 ENAME 을 그대로 SELECT 할 수 없다.

풀이 흐름

- GROUP BY DEPTNO 는 부서별로 행을 묶는다.
- SUM(SAL)은 각 부서 그룹의 급여 합계를 계산하므로 문제가 없다.
- 하지만 ENAME 은 그룹 기준도 아니고 집계 함수도 아니다.
- 한 부서에 사원이 여러 명이면 어떤 ENAME 을 보여줘야 할지 모호하므로 오류가 난다.

비교 정리

SELECT 항목	허용 여부	이유
DEPTNO	허용	GROUP BY 에 포함된 그룹 기준 컬럼
SUM(SAL)	허용	집계 함수 결과
ENAME	불가	그룹 기준도 집계 결과도 아님

함정 포인트

WHERE/HAVING 문제가 아니라 SELECT 목록 문제다. ENAME 까지 보고 싶다면 GROUP BY DEPTNO, ENAME 으로 묶거나 MIN(ENAME), MAX(ENAME)처럼 집계해야 한다.

문제 35. [집계 결과와 공집합] 다음 두 SQL 의 결과로 옳은 것은?

[T_CHAR]

COL1

a

b

c

[SQL1]

```
SELECT COL1 FROM T_CHAR WHERE COL1 = 'z';
```

[SQL2]

```
SELECT MAX(COL1) FROM T_CHAR WHERE COL1 = 'z';
```

- ① SQL1=NULL, SQL2=NULL
- ② SQL1=NULL, SQL2=공집합
- ③ SQL1=공집합, SQL2=NULL
- ④ SQL1=공집합, SQL2=공집합

정답: ③ SQL1=공집합, SQL2=NULL

입문자용 상세 해설

조건을 만족하는 행이 없을 때 일반 SELECT 는 결과 행 자체가 없지만, GROUP BY 없는 집계 함수는 전체 결과를 하나의 그룹으로 보고 1 행을 반환한다. 이때 MAX 처럼 값이 없으면 NULL 이 나온다.

풀이 흐름

- SQL1 은 COL1='z'인 행을 그대로 조회한다. 그런 행이 없으므로 공집합, 즉 0 행이다.
- SQL2 는 COL1='z'인 행들에 대해 MAX(COL1)을 계산한다.
- 대상 행은 없지만 GROUP BY 가 없는 전체 집계는 결과 행을 1 개 반환한다.
- 값이 없으므로 MAX 결과는 NULL 이다.

비교 정리

SQL 형태	조건 만족 행 없음	결과
일반 SELECT	행을 그대로 반환	0 행, 공집합
MAX/MIN/SUM 등 집계, GROUP BY 없음	전체를 하나의 집계 대상으로 처리	1 행, 값은 NULL
COUNT(*), GROUP BY 없음	전체 행 수 계산	1 행, 값은 0

함정 포인트

NULL 과 공집합은 다르다. NULL 은 “값이 비어 있는 1 행”이고, 공집합은 “행이 아예 없음”이다.

문제 36. [GROUP BY 와 NULL 그룹] 다음 SQL 의 결과에서 그룹 수와 DEPTNO 가 NULL 인 그룹의 COUNT(*)로 옳은 것은?

[T_GRP]
DEPTNO
10
10
20
NULL
NULL

[SQL]
SELECT DEPTNO, COUNT(*)
FROM T_GRP
GROUP BY DEPTNO;

- ① 그룹 수=2, NULL 그룹 COUNT=0
- ② 그룹 수=3, NULL 그룹 COUNT=2
- ③ 그룹 수=4, NULL 그룹 COUNT=1
- ④ 그룹 수=5, NULL 그룹 COUNT=2

정답: ② 그룹 수=3, NULL 그룹 COUNT=2

입문자용 상세 해설

GROUP BY 에서는 NULL 값들이 하나의 그룹으로 묶인다. DEPTNO 값은 10, 20, NULL 세 종류로 그룹화되며, NULL 그룹에는 두 행이 있으므로 COUNT(*)는 2 다.

풀이 흐름

- DEPTNO=10 인 행은 2 개이므로 10 그룹이 있다.
- DEPTNO=20 인 행은 1 개이므로 20 그룹이 있다.
- DEPTNO 가 NULL 인 행은 2 개이고, GROUP BY 에서는 이들이 하나의 NULL 그룹으로 묶인다.
- 따라서 그룹 수는 3 개, NULL 그룹의 COUNT(*)는 2 다.

비교 정리

그룹 값	행 수
10	2
20	1
NULL	2

함정 포인트

NULL 은 = 비교에서는 같다고 판단되지 않지만, GROUP BY 에서는 같은 그룹으로 모인다. SQLD 에서 자주 나오는 차이다.

문제 37. [집합 연산자] 다음 데이터에서 각 SQL 의 결과 행 수로 옳은 것은?

[A]
COL
1
1
2
3

[B]
COL
2
2
3
4

[SQL1] SELECT COL FROM A UNION ALL SELECT COL FROM B;
[SQL2] SELECT COL FROM A UNION SELECT COL FROM B;
[SQL3] SELECT COL FROM A INTERSECT SELECT COL FROM B;
① SQL1=8, SQL2=4, SQL3=2
② SQL1=6, SQL2=4, SQL3=2
③ SQL1=8, SQL2=6, SQL3=3
④ SQL1=4, SQL2=4, SQL3=2

정답: ① SQL1=8, SQL2=4, SQL3=2

입문자용 상세 해설

집합 연산자는 두 SELECT 결과를 집합처럼 결합한다. UNION ALL 은 중복을 보존하고, UNION 과 INTERSECT 는 기본적으로 중복을 제거한 결과를 반환한다.

풀이 흐름

- UNION ALL: A 4 행과 B 4 행을 그대로 붙이므로 8 행이다.
- UNION: A 와 B 의 모든 값을 합친 뒤 중복을 제거한다. 값은 1,2,3,4 이므로 4 행이다.
- INTERSECT: 양쪽에 공통으로 존재하는 값을 구한다. 공통 값은 2,3 이므로 2 행이다.

비교 정리

연산자	중복 처리	의미	이 문제 결과 행 수
UNION ALL	보존	두 결과를 그대로 연결	8
UNION	제거	합집합	4
INTERSECT	제거	교집합	2
MINUS	제거	왼쪽 결과에서 오른쪽 결과 제거	문제에는 없음

함정 포인트

UNION 과 UNION ALL 은 성능과 결과가 모두 다를 수 있다. 중복 제거가 필요 없으면 UNION ALL 이 더 단순하고 빠른 경우가 많다.

문제 38. [그룹 함수 - ROLLUP, CUBE] 다음 데이터에 대해 GROUP BY ROLLUP(REGION, PRODUCT)을 수행했을 때 결과 행 수로 옳은 것은?

[SALES]
REGION | PRODUCT | AMT
EAST | A | 100
EAST | B | 200
WEST | A | 300

[SQL]
SELECT REGION, PRODUCT, SUM(AMT)
FROM SALES
GROUP BY ROLLUP(REGION, PRODUCT);

- ① 3
- ② 5
- ③ 6
- ④ 8

정답: ③ 6

입문자용 상세 해설

ROLLUP(REGION, PRODUCT)은 상세 그룹, REGION 별 소계, 전체 합계를 만든다. 실제 데이터에 존재하는 상세 조합은 EAST-A, EAST-B, WEST-A 총 3 개다.

풀이 흐름

- 상세 그룹 (REGION, PRODUCT): EAST-A, EAST-B, WEST-A 로 3 건이다.
- REGION 별 소계: EAST, WEST 로 2 건이다.
- 전체 합계: 모든 데이터를 합친 1 건이다.
- 따라서 결과 행 수는 3+2+1=6 이다.

흐름 도식

ROLLUP(REGION, PRODUCT)
= (REGION, PRODUCT) 상세
+ (REGION) 소계
+ () 전체 합계

비교 정리

그룹 함수	생성하는 집계
ROLLUP(A,B)	(A,B) → (A) → 전체. 계층형 소계에 적합
CUBE(A,B)	(A,B), (A), (B), 전체. 모든 조합 소계
GROUPING SETS	원하는 그룹 조합을 직접 지정

함정 포인트

ROLLUP 은 실제 존재하지 않는 상세 조합까지 억지로 만들지 않는다. WEST-B 데이터가 없으므로 상세 행 WEST-B 는 생기지 않는다.

문제 39. [GROUPING 함수] 다음 SQL 의 결과에서 GROUPING(DEPTNO)=0 이고 GROUPING(JOB)=1 인 행의 의미로 가장 적절한 것은?

```
[SQL]
SELECT DEPTNO, JOB, SUM(SAL),
       GROUPING(DEPTNO) AS G_D,
       GROUPING(JOB) AS G_J
FROM EMP
GROUP BY ROLLUP(DEPTNO, JOB);
```

- ① 부서와 직무별 상세 행
- ② 부서별 소계 행
- ③ 직무별 소계 행
- ④ 전체 합계 행

정답: ② 부서별 소계 행

입문자용 상세 해설

GROUPING(컬럼)은 ROLLUP/CUBE 결과에서 해당 컬럼이 집계 때문에 요약된 컬럼인지 알려준다. 값이 1 이면 그 컬럼이 요약되어 NULL 처럼 표시된 것이고, 0 이면 실제 그룹 기준에 남아 있는 컬럼이다.

풀이 흐름

- GROUPING(DEPTNO)=0 은 DEPTNO 가 그룹 기준으로 남아 있다는 뜻이다.
- GROUPING(JOB)=1 은 JOB 이 집계 과정에서 요약되어 빠졌다는 뜻이다.
- 따라서 특정 부서별로 JOB 전체를 합친 행, 즉 부서별 소계 행이다.

비교 정리

ROLLUP(DEPTNO, JOB) 행 유형	GROUPING(DEPTNO)	GROUPING(JOB)	의미
상세 행	0	0	부서+직무별 합계
부서 소계 행	0	1	부서별 전체 직무 합계
전체 합계 행	1	1	전체 합계

함정 포인트

ROLLUP 결과에서 NULL 이 보일 때 실제 데이터의 NULL 인지, 집계 때문에 생긴 NULL 인지 헷갈릴 수 있다. 이때 GROUPING 함수로 구분한다.

문제 40. [윈도우 함수 - 순위 함수] 다음 결과가 나오도록 빈칸에 들어갈 함수로 가장 적절한 것은?

```
[T_RANK]
NAME | SALARY
BOB | 7000
KIM | 7000
LEE | 6000
CHOI | 5000
```

[SQL]

```
SELECT NAME, SALARY,  
       _____ OVER (ORDER BY SALARY DESC) AS RNK  
FROM T_RANK;
```

[결과]

```
BOB 7000 1  
KIM 7000 1  
LEE 6000 2  
CHOI 5000 3
```

- ① RANK()
- ② DENSE_RANK()
- ③ ROW_NUMBER()
- ④ NTILE(3)

정답: ② DENSE_RANK()

입문자용 상세 해설

동점이 있을 때 공동 순위를 주고, 다음 순위를 건너뛰지 않으면 DENSE_RANK 를 사용한다. 7000 급여가 두 명이므로 두 명 모두 1 등이고, 다음 6000 은 2 등이 된다.

풀이 흐름

- BOB 과 KIM 은 SALARY=7000 으로 공동 1 등이다.
- DENSE_RANK 는 공동 순위 다음 번호를 연속해서 부여하므로 LEE 는 2 등이다.
- CHOI 는 그다음 급여이므로 3 등이다.
- RANK 라면 공동 1 등 다음 순위를 3 으로 건너뛰므로 결과가 다르다.
- ROW_NUMBER 는 동점이어도 각 행에 서로 다른 번호를 부여한다.

비교 정리

함수	동점 처리	이 데이터의 순위 예시
ROW_NUMBER()	동점이어도 고유 번호 부여	1, 2, 3, 4
RANK()	공동 순위 후 다음 번호를 건너뛴	1, 1, 3, 4
DENSE_RANK()	공동 순위 후 다음 번호를 건너뛰지 않음	1, 1, 2, 3
NTILE(n)	정렬된 행을 n 개 그룹으로 나눔	순위보다 버킷 분할에 가까움

함정 포인트

OVER 안의 ORDER BY 는 순위 계산 기준이다. 최종 출력 순서까지 보장하려면 SELECT 문 끝에 ORDER BY SALARY DESC, NAME 같은 정렬 절을 추가하는 것이 안전하다.

문제 41. [KEEP(DENSE_RANK FIRST/LAST)] 다음 SQL 의 결과로 옳은 것은?

```
[T_KEEP]
COL1 | COL2
1   | 10
1   | 20
2   | 30
3   | 40
3   | 50
```

```
[SQL]
SELECT MAX(COL2) KEEP (DENSE_RANK FIRST ORDER BY COL1) AS R
FROM T_KEEP;
```

- ① 10
- ② 20
- ③ 40
- ④ 50

정답: ② 20

입문자용 상세 해설

KEEP(DENSE_RANK FIRST ORDER BY COL1)은 COL1 기준으로 가장 첫 번째 순위에 해당하는 행 집합을 고른 뒤, 그 집합 안에서 집계 함수를 적용한다. COL1 이 가장 작은 값은 1 이고, 그 행들의 COL2 는 10 과 20 이다. MAX(COL2)는 20 이다.

풀이 흐름

- ORDER BY COL1 기준으로 가장 작은 COL1 은 1 이다.
- COL1=1 인 행은 두 개이고 COL2 값은 10, 20 이다.
- KEEP 은 이 첫 순위 집합을 대상으로 삼는다.
- 그 집합 안에서 MAX(COL2)를 계산하면 20 이다.

비교 정리

표현	의미	이 데이터에서 결과
MAX(COL2) KEEP (DENSE_RANK FIRST ORDER BY COL1)	COL1 최소 그룹에서 COL2 최대값	20
MIN(COL2) KEEP (DENSE_RANK FIRST ORDER BY COL1)	COL1 최소 그룹에서 COL2 최소값	10
MAX(COL2) KEEP (DENSE_RANK LAST ORDER BY COL1)	COL1 최대 그룹에서 COL2 최대값	50
DENSE_RANK() OVER (...)	행마다 순위를 계산하는 분석 함수	행별 순위 반환

함정 포인트

KEEP 안의 DENSE_RANK FIRST/LAST 는 “순위를 출력”하는 것이 아니라, 집계할 대상 행 집합을 고르는 역할이다.

문제 42. [윈도우 함수 - ROWS 와 RANGE] 다음 SQL 에서 NAME=A 인 행의 CNT 값으로 옳은 것은?

[T_PRICE]
NAME | PRICE
A | 100
B | 100
C | 200
D | 300

[SQL]
SELECT NAME, PRICE,
COUNT(*) OVER (
ORDER BY PRICE
RANGE BETWEEN CURRENT ROW AND CURRENT ROW
) AS CNT
FROM T_PRICE;

- ① 1
- ② 2
- ③ 3
- ④ 4

정답: ② 2

입문자용 상세 해설

윈도우 프레임에서 ROWS 는 물리적인 행 수 기준이고, RANGE 는 ORDER BY 값의 범위 기준이다. RANGE CURRENT ROW 는 현재 행과 ORDER BY 값이 같은 동물 행까지 같은 프레임으로 본다.

풀이 흐름

- NAME=A 인 행의 PRICE 는 100 이다.
- ORDER BY PRICE 기준으로 PRICE=100 인 행은 A 와 B 두 개다.
- RANGE BETWEEN CURRENT ROW AND CURRENT ROW 는 PRICE 값이 현재 행과 같은 모든 행을 포함한다.
- 따라서 A 행에서 COUNT(*)는 A 와 B 를 세어 2 가 된다.

비교 정리

프레임 방식	기준	CURRENT ROW 의미
ROWS	물리적 행 위치	현재 행 1 건
RANGE	ORDER BY 값의 범위	현재 행과 같은 정렬값을 가진 동물 행 포함 가능

함정 포인트

ORDER BY 컬럼 값이 중복될 때 ROWS 와 RANGE 결과가 달라질 수 있다. 동점 데이터를 다루는 윈도우 함수 문제에서 자주 나오는 함정이다.

문제 43. [윈도우 함수 - NTILE] 다음 SQL 에서 NT=3 인 그룹의 행 수로 옳은 것은?

[T_NTILE]

ID | SCORE

1 | 10

2 | 20

3 | 25

4 | 30

5 | 40

6 | 50

7 | 60

8 | 70

[SQL]

```
SELECT NT, COUNT(*)
FROM (
  SELECT ID, SCORE, NTILE(3) OVER (ORDER BY SCORE) AS NT
  FROM T_NTILE
)
GROUP BY NT;
```

① 1

② 2

③ 3

④ 4

정답: ② 2

입문자용 상세 해설

NTILE(3)은 정렬된 전체 행을 가능한 균등하게 3 개 그룹으로 나눈다. 8 건을 3 그룹으로 나누면 기본적으로 각 그룹에 2 건씩 들어가고, 남은 2 건은 앞 그룹부터 하나씩 추가된다.

풀이 흐름

- $8 \div 3 = \text{몫 } 2, \text{ 나머지 } 2$ 다.
- 모든 그룹에 기본 2 건씩 배정한다.
- 나머지 2 건은 1 번 그룹과 2 번 그룹에 각각 1 건씩 추가한다.
- 따라서 그룹 크기는 3, 3, 2 다.
- NT=3 인 그룹은 2 건이다.

흐름 도식

8 행을 3 개 버킷으로 분배

기본: 2, 2, 2

나머지 2 개를 앞에서부터 배정

최종: 3, 3, 2

비교 정리

버킷	행 수
NT=1	3
NT=2	3
NT=3	2

함정 포인트

NTILE 은 점수 구간을 같은 폭으로 나누는 함수가 아니라, 정렬된 행 개수를 균등하게 나누는 함수다.

문제 44. [Top N Query - ROWNUM] Oracle 기준으로 급여 상위 3 명을 올바르게 조회하는 SQL 은?

- ① SELECT ENAME, SAL FROM EMP WHERE ROWNUM <= 3 ORDER BY SAL DESC;
- ② SELECT ENAME, SAL FROM (SELECT ENAME, SAL FROM EMP ORDER BY SAL DESC) WHERE ROWNUM <= 3;
- ③ SELECT ENAME, SAL FROM EMP WHERE ROWNUM = 3 ORDER BY SAL DESC;
- ④ SELECT ENAME, SAL FROM EMP WHERE ROWNUM > 3 ORDER BY SAL DESC;

정답: ② SELECT ENAME, SAL FROM (SELECT ENAME, SAL FROM EMP ORDER BY SAL DESC) WHERE ROWNUM <= 3;

입문자용 상세 해설

Oracle 의 ROWNUM 은 같은 SELECT 블록에서 ORDER BY 보다 먼저 부여된다. 따라서 급여순으로 정렬한 뒤 상위 3 명을 가져오려면 인라인 뷰에서 먼저 ORDER BY 를 수행하고, 바깥 SELECT 에서 ROWNUM <= 3 을 적용해야 한다.

풀이 흐름

- 1 번 보기처럼 WHERE ROWNUM <= 3 을 먼저 적용하면 정렬 전 임의의 3 건을 가져온 뒤 그 3 건만 급여순 정렬할 수 있다.
- 2 번 보기는 인라인 뷰 내부에서 EMP 를 급여 내림차순으로 정렬한다.
- 바깥 SELECT 에서 이미 정렬된 결과 중 앞 3 건에 ROWNUM <= 3 을 적용한다.
- 따라서 급여 상위 3 명을 올바르게 조회한다.
-

비교 정리

방식	설명	Top N 적합성
ROWNUM 단독	같은 SELECT 블록에서 정렬보다 먼저 부여될 수 있음	주의 필요
인라인 뷰 + ROWNUM	안쪽에서 정렬, 바깥에서 제한	Oracle 전통적 Top N
ROW_NUMBER() 분석 함수	정렬 기준으로 행 번호 계산	파티션별 Top N 에 유용
OFFSET/FETCH	정렬 후 행 제한 문법	최신 문법, 페이지 조회에 유용

함정 포인트

ROWNUM = 3 또는 ROWNUM > 3 은 일반적인 상위 N 조회 방식으로 동작하지 않는다. ROWNUM 은 1 번 행이 먼저 통과해야 다음 번호가 만들어진다.

문제 45. [Top N Query - OFFSET/FETCH] 다음 SQL 의 결과로 출력되는 NO 값의 목록으로 옳은 것은?

[T_NO]

NO

1

2

3

4

5

6

7

8

9

10

[SQL]

```
SELECT NO
FROM T_NO
ORDER BY NO DESC
OFFSET 3 ROWS
FETCH FIRST 4 ROWS ONLY;
```

① 10, 9, 8, 7

② 7, 6, 5, 4

③ 6, 5, 4, 3

④ 4, 3, 2, 1

정답: ② 7, 6, 5, 4

입문자용 상세 해설

OFFSET 은 정렬된 결과에서 앞의 행을 건너뛰는 기능이고, FETCH FIRST 는 그다음 몇 행을 가져올지 지정한다. 먼저 NO 를 내림차순 정렬한 뒤 앞 3 행을 건너뛰고 4 행을 가져온다.

풀이 흐름

- ORDER BY NO DESC 결과는 10, 9, 8, 7, 6, 5, 4, 3, 2, 1 이다.
- OFFSET 3 ROWS 는 앞의 10, 9, 8 세 행을 건너뛴다.
- FETCH FIRST 4 ROWS ONLY 는 그다음 7, 6, 5, 4 네 행을 반환한다.

흐름 도식

10, 9, 8, 7, 6, 5, 4, 3, 2, 1

[건너뛴] 10, 9, 8

[반환] 7, 6, 5, 4

비교 정리

구문	의미
ORDER BY	반환 순서 결정
OFFSET n ROWS	앞에서 n 행 건너뛴
FETCH FIRST m ROWS ONLY	그다음 m 행 반환

함정 포인트

페이지 조회에서는 ORDER BY 가 필수에 가깝다. 정렬 기준이 없으면 “앞의 3 행”과 “다음 4 행”의 의미가 안정적이지 않다.

문제 46. [계층형 질의] 다음 부서 계층에서 DEPT_ID=1 인 부서를 루트로 하위 부서를 전개하려고 한다.

Oracle 계층형 질의의 CONNECT BY 조건으로 가장 적절한 것은?

```
[DEPT]
DEPT_ID | PARENT_DEPT_ID
1       | NULL
2       | 1
3       | 1
4       | 2
```

[목표]

1 → 2 → 4, 그리고 1 → 3 방향으로 하위 전개

- ① START WITH DEPT_ID = 1 CONNECT BY PRIOR DEPT_ID = PARENT_DEPT_ID
- ② START WITH DEPT_ID = 1 CONNECT BY DEPT_ID = PRIOR PARENT_DEPT_ID
- ③ START WITH PARENT_DEPT_ID = 1 CONNECT BY PRIOR PARENT_DEPT_ID = DEPT_ID
- ④ START WITH DEPT_ID = 4 CONNECT BY PRIOR DEPT_ID = PARENT_DEPT_ID

정답: ① START WITH DEPT_ID = 1 CONNECT BY PRIOR DEPT_ID = PARENT_DEPT_ID

입문자용 상세 해설

Oracle 계층형 질의에서 START WITH 는 시작 루트 행을 지정하고, CONNECT BY 는 부모와 자식의 연결 조건을 지정한다. 부모에서 자식 방향으로 내려가려면 부모의 DEPT_ID 가 자식의 PARENT_DEPT_ID 와 같아야 한다.

풀이 흐름

- 루트는 DEPT_ID=1 이므로 START WITH DEPT_ID = 1 이다.
- 현재 부모 행의 DEPT_ID 를 자식 행의 PARENT_DEPT_ID 와 비교해야 한다.
- PRIOR DEPT_ID 는 “이전 단계, 즉 부모 행의 DEPT_ID”를 의미한다.
- 따라서 CONNECT BY PRIOR DEPT_ID = PARENT_DEPT_ID 가 하위 전개 조건이다.
- 결과적으로 1 의 자식 2,3 이 나오고, 2 의 자식 4 가 나온다.
-

흐름 도식

```
1
├─ 2
│  └─ 4
└─ 3
```

비교 정리

구문	역할
START WITH	계층 탐색을 시작할 루트 행 지정
CONNECT BY	부모-자식 연결 조건 지정
PRIOR	이전 단계의 행, 즉 부모 또는 자식 방향을 결정하는 기준

함정 포인트

PRIOR 의 위치가 방향을 결정한다. PRIOR 가 부모 컬럼 쪽에 붙으면 부모에서 자식으로 내려가고, 반대로 쓰면 자식에서 부모로 올라가는 형태가 된다.

문제 47. [셀프 조인] 사원과 그 관리자의 이름을 함께 조회하되, 관리자가 없는 최상위 사원도 출력하려고 한다. 가장 적절한 SQL 은?

[EMP]

EMP_ID | ENAME | MGR_ID

- ① SELECT E.ENAME, M.ENAME FROM EMP E INNER JOIN EMP M ON E.MGR_ID = M.EMP_ID;
- ② SELECT E.ENAME, M.ENAME FROM EMP E LEFT OUTER JOIN EMP M ON E.MGR_ID = M.EMP_ID;
- ③ SELECT E.ENAME, M.ENAME FROM EMP E LEFT OUTER JOIN EMP M ON E.EMP_ID = M.MGR_ID;
- ④ SELECT E.ENAME, M.ENAME FROM EMP E CROSS JOIN EMP M;

정답: ② SELECT E.ENAME, M.ENAME FROM EMP E LEFT OUTER JOIN EMP M ON E.MGR_ID = M.EMP_ID;

입문자용 상세 해설

셀프 조인은 같은 테이블을 두 번 사용하는 조인이다. 이 문제에서는 EMP 를 사원 역할 E 와 관리자 역할 M 으로 나누어 생각해야 한다. 사원의 MGR_ID 가 관리자의 EMP_ID 를 참조한다.

풀이 흐름

- E 는 사원 행을 의미한다.
- M 은 관리자 행을 의미한다. 실제로는 같은 EMP 테이블이지만 별칭을 다르게 둔다.
- 사원 E 의 MGR_ID 와 관리자 M 의 EMP_ID 가 같아야 한다.
- 관리자가 없는 최상위 사원도 출력해야 하므로 E 를 기준으로 LEFT OUTER JOIN 을 사용한다.

흐름 도식

EMP E(사원) EMP M(관리자)
E.MGR_ID —————→ M.EMP_ID

비교 정리

보기의 방향	의미
E.MGR_ID = M.EMP_ID	사원의 관리자 찾기
E.EMP_ID = M.MGR_ID	사원의 부하 직원 찾기
INNER JOIN	관리자가 없는 사원 제외
LEFT OUTER JOIN	관리자가 없는 사원도 보존

함정 포인트

셀프 조인에서는 별칭의 역할을 먼저 정해야 한다. E 가 사원인지, M 이 관리자인지 정하지 않으면 ON 조건을 반대로 쓰기 쉽다.

문제 48. [PIVOT] Oracle 기준으로 다음 PIVOT SQL 을 수행했을 때 SEMESTER='2025-1' 행의 MATH, ENG 값으로 옳은 것은?

[S_ENR]
STUDENT_ID | COURSE | SEMESTER
S001 | MATH | 2025-1
S002 | ENG | 2025-1
S003 | MATH | 2025-1
S004 | ENG | 2025-2
S005 | SCI | 2025-2

[SQL]
SELECT *
FROM (SELECT SEMESTER, COURSE FROM S_ENR)
PIVOT (
COUNT(*) FOR COURSE IN ('MATH' AS MATH, 'ENG' AS ENG, 'SCI' AS SCI)
);
① MATH=1, ENG=1
② MATH=2, ENG=1
③ MATH=2, ENG=2
④ MATH=0, ENG=1

정답: ② MATH=2, ENG=1

입문자용 상세 해설

PIVOT 은 행으로 있던 값을 컬럼으로 돌려서 집계하는 기능이다. 이 문제에서는 COURSE 값인 MATH, ENG, SCI 가 각각 컬럼이 되고, SEMESTER 별로 COUNT(*)가 계산된다.

풀이 흐름

- PIVOT 전에는 2025-1 학기에 MATH 가 S001, S003 두 건 있다.
- 2025-1 학기에 ENG 는 S002 한 건 있다.
- PIVOT 의 COUNT(*)는 COURSE 별 건수를 센다.
- 따라서 2025-1 행의 MATH=2, ENG=1 이다.

비교 정리

기능	방향	예시
PIVOT	행 값 → 컬럼	COURSE 값 MATH/ENG/SCI 를 컬럼으로 전환
UNPIVOT	컬럼 → 행 값	KOR/ENG/MATH 컬럼을 SUBJECT 행으로 전환
조건부 집계	CASE + GROUP BY 로 PIVOT 유사 구현	SUM(CASE WHEN COURSE='MATH' THEN 1 ELSE 0 END)

함정 포인트

PIVOT 은 단순한 모양 변경이 아니라 집계를 포함한다. PIVOT 대상에 포함되지 않은 컬럼은 묵시적인 그룹 기준이 된다.

문제 49. [UNPIVOT] Oracle 기준으로 다음 UNPIVOT SQL 의 결과 행 수로 옳은 것은?

```
[SCORE_WIDE]
STUDENT | KOR | ENG | MATH
S1      | 90 | NULL | 80
S2      | NULL | 70 | 60
```

```
[SQL]
SELECT STUDENT, SUBJECT, SCORE
FROM SCORE_WIDE
UNPIVOT (
  SCORE FOR SUBJECT IN (KOR, ENG, MATH)
)
```

- ① 3
- ② 4
- ③ 5
- ④ 6

정답: ② 4

입문자용 상세 해설

UNPIVOT 은 여러 컬럼을 하나의 컬럼 값으로 세로 변환하는 기능이다. Oracle UNPIVOT 은 기본적으로 NULL 값을 제외하므로 점수가 NULL 인 과목은 결과 행으로 나오지 않는다.

풀이 흐름

- S1 은 KOR=90, ENG=NULL, MATH=80 이다. NULL 인 ENG 는 제외되어 2 행이 나온다.
- S2 는 KOR=NULL, ENG=70, MATH=60 이다. NULL 인 KOR 는 제외되어 2 행이 나온다.
- 총 결과 행 수는 2+2=4 다.
- 만약 INCLUDE NULLS 를 명시하면 S1 의 ENG 와 S2 의 KOR 까지 포함되어 6 행이 된다.

비교 정리

학생	기본 UNPIVOT 결과에 포함되는 과목	행 수
S1	KOR, MATH	2
S2	ENG, MATH	2
합계	-	4

함정 포인트

PIVOT 은 행을 열로 넓히는 방향이고, UNPIVOT 은 열을 행으로 길게 만드는 방향이다. 기본 UNPIVOT 은 NULL 을 제외한다는 점을 꼭 기억해야 한다.

문제 50. [날짜 연산] Oracle 기준으로 다음 SQL 의 결과로 옳은 것은?

[SQL]

```
SELECT TO_CHAR(DATE '2025-01-01' + 1/24 + 1/24/60,
              'YYYY-MM-DD HH24:MI:SS') AS R
FROM DUAL;
```

- ① 2025-01-01 00:01:00
- ② 2025-01-01 01:00:00
- ③ 2025-01-01 01:01:00
- ④ 2025-01-02 01:01:00

정답: ③ 2025-01-01 01:01:00

입문자용 상세 해설

Oracle DATE 에 숫자를 더하면 “일(day)” 단위로 더해진다. 따라서 1 은 1 일, 1/24 는 1 시간, 1/24/60 은 1 분이다.

풀이 흐름

- DATE '2025-01-01'은 2025 년 1 월 1 일 00:00:00 을 의미한다.
- 1/24 는 하루를 24 로 나눈 값이므로 1 시간이다.
- 1/24/60 은 하루를 24 시간과 60 분으로 나눈 값이므로 1 분이다.
- 기준 날짜에 1 시간 1 분을 더하면 2025-01-01 01:01:00 이다.
- TO_CHAR 는 날짜 값을 'YYYY-MM-DD HH24:MI:SS' 형식의 문자열로 보여준다.

비교 정리

표현	의미	시간 환산
1	1 일	24 시간
1/24	1 시간	60 분
1/24/60	1 분	60 초
1/60	1 분이 아니라 1/60 일	24 분

함정 포인트

날짜 + 숫자에서 숫자는 분이냐 시간이 아니라 “일” 기준이다. 그래서 1/60 은 1 분이 아니라 24 분이다. 가독성을 높이려면 INTERVAL 표현을 쓰는 방법도 있다.

마무리 암기 체크리스트

- SELECT 논리 실행 순서: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
- 논리 연산자 우선순위: NOT → AND → OR
- COUNT(*)는 NULL 포함, COUNT(컬럼)은 NULL 제외, COUNT(DISTINCT 컬럼)은 중복 제거
- NULL 비교는 = NULL 이 아니라 IS NULL / IS NOT NULL 을 사용
- NOT IN 하위 질의에 NULL 이 있으면 결과가 0 건이 될 수 있음
- OUTER JOIN 에서 오른쪽 테이블 조건을 WHERE 에 두면 기준 행이 사라질 수 있음
- NATURAL JOIN/USING 공통 컬럼은 Oracle 에서 별칭으로 한정하지 않음
- ROLLUP 은 계층형 소계, CUBE 는 모든 조합 소계, GROUPING 은 집계 NULL 구분
- RANK 는 순위 건너뛰, DENSE_RANK 는 건너뛰지 않음, ROW_NUMBER 는 고유 번호
- ROWNUM Top N 은 정렬을 인라인 뷰에서 먼저 수행한 뒤 바깥에서 제한
- UNPIVOT 은 기본적으로 NULL 제외, INCLUDE NULLS 를 쓰면 NULL 포함
- Oracle DATE + 숫자에서 숫자 1 은 1 일, 1/24 는 1 시간, 1/24/60 은 1 분